VIBE CODING 教學案例 02

東京環狀 24時
捲動穿越的 3D 藝廊

判斷什麼問題該用什麼技術。
每一個工具都要說得出它在解決什麼問題，以及不用它會怎樣。

成品　一個捲動穿越的 3D 照片藝廊：十張東京街頭的照片沿深度排開，從清晨 04:52 到深夜 00:40。捲動時鏡頭穿過它們，背景顏色是從每張照片本身算出來的，點一張會從原位飛出來展開成資訊面板，捲到底無縫接回開頭。

使用技術　Three.js · GSAP · 原生 CSS · importmap（教學版）→ Vite（正式版）

時間　閱讀約 2.6 小時，自己動手約 8.6 小時

對象　沒有程式背景，不需要看懂 shader 或 JavaScript

完成等級　階段 13 = Learning Prototype / 階段 14 = Production-ready Baseline

核心　套件替你寫掉了程式，每一幀的計算量還是照算

案例二：東京環狀 24 時

判斷什麼問題該用什麼技術

> 這份手冊是寫給沒有程式背景的人的。
>
> 你不需要看懂 shader，也不需要會寫 JavaScript。手冊裡會出現一些程式碼，但它們的角色是**證據**，不是要你背下來的東西，每一段下面都有一句「你要看的是什麼」。

這個案例要回答的問題

> 我知道想做什麼之後，要怎麼判斷該用哪一種技術？

這個案例處理的是選對工具。

每一個套件都要問這三個問題：

> 這個效果為什麼要用這個套件？不用會怎樣？用錯會慢在哪裡？

目錄

開始之前

- 學員路線圖
- 這個案例做到最後，完成的是什麼？
- 這個專案是怎麼被建起來的
- 全課地圖
- 開始之前
  - 你需要準備的東西
  - 這個案例需不需要安裝東西
  - 環境檢查（五分鐘）
  - 認識 Claude Code Desktop 的畫面
  - 接上 GitHub（階段 14 才會用到）
  - 課前作業：選十張照片

- 這份手冊怎麼用

隨時可以翻回來查的

- 四張操作卡
  - 操作卡 1 和 AI 一起做事的七個步驟
  - 操作卡 2 哪些事 AI 可以自己做，哪些要先問你
  - 操作卡 3 一個能用的指令，有六個構件
  - 操作卡 4 卡住的時候怎麼辦
- 不會寫程式，怎麼審查 AI 的產出

階段

1. 先看成品，觀察三件之後會用到的事
2. 建立判斷框架：離散事件與連續訊號
   - Checkpoint 1（做完階段 2）
3. 建立不用安裝就能跑的專案
4. 準備照片，並從照片算出背景顏色
5. 建立主迴圈，之後每個模組都掛在它上面
6. 做背景：顏色跟著捲動位置變
   - Checkpoint 2（做完階段 6）
7. 做捲動訊號：哪一半交給套件、哪一半自己寫
8. 讓照片無限循環，而且看不出接縫
9. 讓介面顏色自己跟著背景亮度翻轉
10. 點一張照片，讓它從原位飛到面板

- Checkpoint 3（做完階段 10）

11. 量一次效能，決定哪些其實不用改
12. 補回鍵盤操作、減少動態與觸控判斷
13. 盤點外部依賴，判斷能不能上線
14. 把教學版整理成可以交付的正式版

做完之後

- 完成清單
- 離開這個範例，我會不會自己做？

- 常見卡關
- 為什麼「懂原理」還是重要
- 名詞白話對照表
- 各階段預估時間

🗺️ 學員路線圖

起點	一個空資料夾、十張你自己選的照片
過程	先決定每件事交給套件還是自己算。再依序疊上資料層、主迴圈、背景、照片、介面與面板。最後量效能、換掉外部依賴
產出	一個捲動穿越的 3D 照片藝廊，以及一份依賴風險表 `deps-notes.md`
核心判斷	這個訊號是離散還是連續？離散（有明確的開始和結束）交給套件，連續（每一幀都在變）自己算。階段 2 建立，之後每個階段各用一次

這個案例需要你自己判斷的，只有最後那一列。 程式碼看不懂不影響。

> **關於「階段」**：階段是這份手冊的章節單位。這個案例共 14 個，照順序做。

這個案例做到最後，完成的是什麼？

這個案例有兩個終點，做到哪裡就是哪一級。

```
階段 13 結束 ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓  Learning Prototype（學習用原型）
階段 14 結束 ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓  Production-ready Baseline（可以當正式版的起點）
```

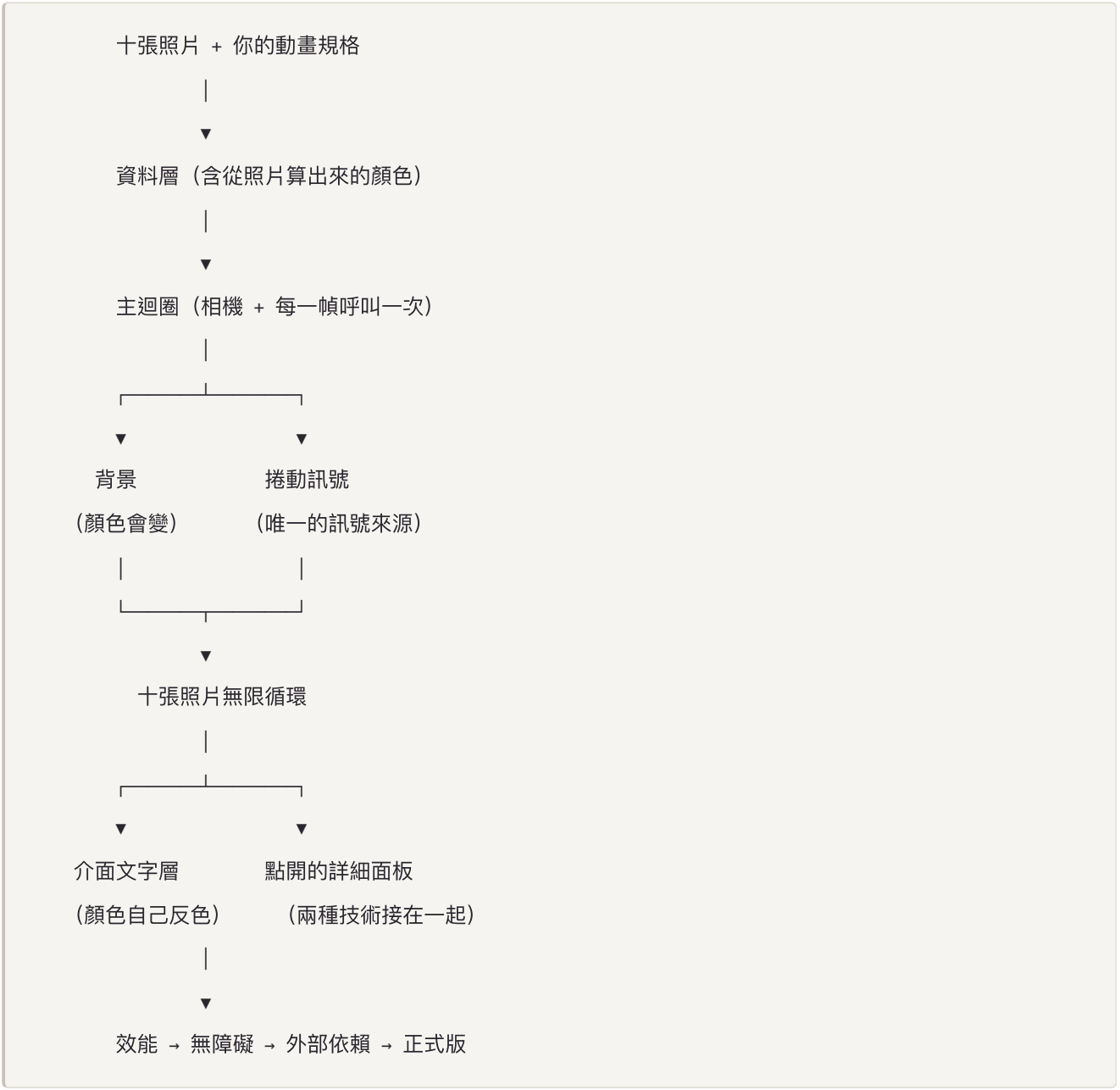
為什麼階段 13 還不能上線

這個版本的照片、字體、程式套件全部來自三個外部網站。任何一個掛掉，畫面就殘缺，其中一個掛掉的話，整個作品根本不會啟動。教學用沒問題，對外就不行。

階段 14 就是專門處理這件事的。它會把三個外部依賴收進你自己的專案，再加上打包步驟。打包是把檔案先組裝壓縮過再上線，〈開始之前〉會解釋。最後真機測過、效能量過。做完才算走到 Production-ready Baseline。

做完 Production-ready Baseline 之後仍然要另外處理的：照片的授權、網站的分析與監控、以及你自己的網域。

這個專案是怎麼被建起來的



全課地圖

階段	產出	重點
1	三個觀察題的答案 + 一張狀態表	打開成品，觸發每個狀態，記下看到的變化
2	AI 對四個選型問題的回答 + 狀態表的分工	丟四個選型問題給 AI，再把狀態表一列一列分給引擎
3	一個會轉的方塊 + 鎖死版本的設定	寫 importmap，四個版本全部鎖死
4	`scenes.js` 與十組算出來的顏色	跑抽色工具，色碼貼進 scenes.js
5	`Engine.js` 與主迴圈	建主迴圈，把解析度上限設成 2
6	會跟著捲動變色的背景	掛上背景模組，每一幀覆寫三個顏色
7	唯一的捲動訊號來源 `Scroll.js` + 引擎分工的自動檢查	接 Observer，自己寫阻尼與速度，再把分界線寫成檢查
8	看不出接縫的無限循環	取餘數折回位置，設半透明三件套
9	會自己反色的文字層	用背景亮度切 class，混色調十個色票
10	從原位飛出來的詳細面板	量起點矩形，照三步順序串起 Flip
11	`perf-notes.md` 效能清單	錄五秒開發者工具，逐項核對清單
12	鍵盤操作、減少動態、觸控判斷	補鍵盤事件，拿真的手機滑十次
13	`deps-notes.md` 依賴風險表	在開發者工具擋掉三個網域，各看一次
14	打包後的正式版	換成 Vite 打包，部署後再量一次

每一個動作背後的判斷，寫在各階段的「為什麼這樣做」。

開始之前

你需要準備的東西

項目	說明
Claude Code Desktop	桌面版應用程式，到 claude.com/download 下載安裝，用你的 Claude 帳號登入。整個案例都在裡面完成
一個付費的 Claude 方案	**免費方案打不開 Code 分頁。** 需要 Pro、Max、Team 或 Enterprise 其中一種。上課前先確認
Windows 使用者多一步	第一次打開 Code 分頁之前要先裝 Git for Windows，裝完把應用程式關掉重開。macOS 不用
Node.js	見下面〈這個案例需不需要安裝東西〉
十張你自己選的照片	課前作業，見下面
一個 GitHub 帳號	只有階段 14 部署會用到。先去 github.com 註冊好，再照下面〈接上 GitHub〉登入一次。Vercel 用同一個帳號登入，不用另外註冊
課程附件	到課程頁面下載 `case-02-tokyo-loop.zip`，解壓縮到桌面。階段 1 會用

> **名詞：** `CLAUDE.md` 是放在專案資料夾裡的一份說明書，每次開新對話，Claude 都會先讀它。
> 你需要知道到什麼程度：知道規則寫在這裡，下一個對話才會照做。這個案例在階段 7 建。

這個案例需不需要安裝東西

這件事本身就是這個案例的第一個技術選型判斷，所以要講清楚。

	需不需要安裝
這個網站本身	**不用。** 用一個叫 importmap 的做法直接從網路載入需要的程式庫，沒有安裝步驟、沒有打包步驟。importmap 階段 3 會說明
階段 4 的抽色工具	要裝一個讀圖片用的小工具。它只放在 `tools/` 資料夾，**不會變成網站的一部分**
階段 14 的正式版整理	要裝打包工具

> **名詞**：**打包工具**是一個在你電腦上跑的程式。它把你寫的一堆檔案合併、壓縮成幾個檔案，再交給瀏覽器。
> 你需要知道到什麼程度：知道它讓網站載得比較快，但每改一次就要多跑一次它。階段 14 會裝一個（Vite）。

> 「零安裝」指的是網站的依賴，不是你的電腦什麼都不用裝。一種是**使用者載入你的網站時要拿到的東西**，另一種是**你在自己電腦上用的工具**。前者每多一個，使用者就多等一次；後者裝多少都不影響使用者。

你的電腦要有 Node.js 22.12 以上。下面〈環境檢查〉會確認，沒有的話照那裡的方法裝。

> **名詞**：**Node.js** 是讓程式碼可以在你自己的電腦上執行的環境。階段 4 的抽色工具、階段 7 的自動檢查、階段 14 的打包工具都靠它。
> 你需要知道到什麼程度：裝好就好，不用學它，它也不會變成你網站的一部分。

環境檢查（五分鐘）

先做完這一步再開始。 環境沒弄好就往下走，後面每一步都會出錯，而且錯誤訊息看起來跟環境無關。

動手

打開 Claude Code Desktop，按左上角的 **Code** 分頁，開一個新對話。**Project folder** 隨便選一個空資料夾就好。它在輸入框上方那一排，位置見下一節的定位圖。把這段貼進去：

貼給 AI

你只需要修改：沒有。整段照貼，四個問題一個都不要刪。

```
幫我檢查這台電腦能不能做網站開發。請逐項回報，先不要幫我安裝任何東西：

1. Node.js 有沒有裝？版本是多少？（我需要 22.12 以上）
2. npm 能不能正常執行？
3. 我對目前這個資料夾有沒有寫入權限？
4. 能不能連到 npm 的伺服器？（隨便查一個套件的版本就好）

每一項都用「可以」或「不行」開頭。
不行的話告訴我：問題是什麼、對一個沒有程式背景的人來說最簡單的解法是什麼。
```

四項都「可以」就準備好了。

常見狀況

訊息	意思	怎麼辦
`command not found: npm`	這台電腦沒有裝 Node.js	到 nodejs.org 下載 **LTS** 版本（長期支援版，比較穩），一路按下一步
版本是 18、20 這種數字	太舊了	同上，重裝一次 LTS 版本
Windows 上 Code 分頁打不開	少了 Git for Windows	到 git-scm.com 安裝，裝完把應用程式重開
`EACCES` 或 `permission denied`	那個資料夾不讓你寫東西	換一個資料夾，例如在桌面上新建一個
安裝一直卡住不動	網路擋住了 npm	換一個網路（手機熱點通常可以）。公司電腦的話問一下 IT
Views 選單裡沒有 Browser 面板	應用程式版本太舊	選單列的 **Claude → Check for Updates**（Windows 是 **Help → Check for Updates**）
硬碟空間不足	一個專案大約要 200–300MB	清一些空間再來

認識 Claude Code Desktop 的畫面

打開應用程式之後，**視窗左上角有 Home 和 Code 兩個分頁。這個案例只會用到 Code 分頁。**

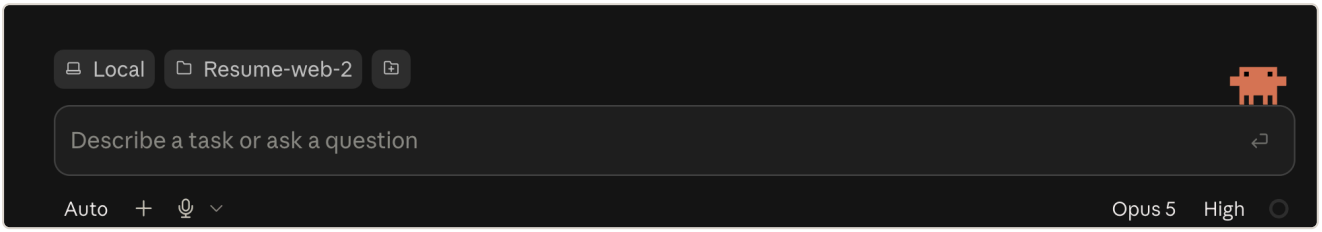
（`Chat` 和 `Cowork` 不是分頁。它們是 Home 畫面輸入框右下角的兩個切換鈕，這個案例不會用到。）

介面定位圖：主要會碰到六個地方

主要會用到下面六個位置，找不到某一項時回來看這張表。Views 選單裡還有一個 Terminal，只有手冊寫「你自己打」的指令才會用到。



> **你要看的是什麼**：① 在**視窗最左上角**，和 Home 並排。打開應用程式時會停在 **Home**，**每次都要先按一下 Code**，這份手冊講的所有東西才會出現在你眼前。



> **你要看的是什麼**：② ③ ④ 三個設定全部集中在**視窗最下方這一條**，不在任何選單裡。左上角的 `Local` 是 Environment。旁邊的資料夾名稱（圖上是 `Resume-web-2`）是 Project folder，再旁邊那個小按鈕可以換資料夾。輸入框**下面**那一排的最左邊（圖上是 `Auto`）就是權限模式選單。大部分時間要把它切成 **Plan** 或 **Manual**。右下角的 `Opus 5` 是模型，用預設的就好。

#	名稱	在哪裡找	用途
①	**Code**（分頁）	視窗**左上角**，和 Home 並排（見上面第一張圖）	整個案例只待在這個分頁。開啟時停在 Home，要先按 Code
②	**Environment**	輸入框**上方**那一排的第一個（上圖是 `Local`）	選 **Local**，檔案才會建在你自己的電腦上
③	**Project folder**	緊接在 Environment 右邊（上圖是 `Resume-web-2`）	指定 Claude 只能在哪個資料夾裡工作
④	**模式選單**（Plan / Manual …）	輸入框**下方**那一排的最左邊（上圖是 `Auto`），隨時可切	手冊寫「切到 Plan」的地方先用 **Plan** 看計畫，確認後切 **Manual**。其餘直接用 **Manual**

#	名稱	在哪裡找	用途
⑤	**Views → Browser**	選單列的 **Views**（上圖沒拍到）	直接預覽你做的網站，可以拖到對話框旁邊並排
⑥	**Views → Diff**	同一個 Views 選單（上圖沒拍到）	看 Claude 這次改了哪幾行。**這是你最主要的驗收工具**

開始一個對話之前，先設定三件事

位置都見上面的定位圖，這裡只講要選什麼：

1. **Environment** 選 `Local`
2. **Project folder** 選那一階段手冊指定的資料夾
3. **模式選單** 見下一段

模型用預設的就好。

權限模式：決定 AI 什麼時候要先問你

模式選單在輸入框**下方**那一排的最左邊（見上面的介面定位圖），隨時可以切換。

模式	白話	用在哪
Plan	只規劃，不動手	手冊寫「切到 Plan」的地方用它，先看 Claude 打算怎麼做
Manual	每一個改動都先問你	確認計畫之後執行用這個。其餘時間都用它
Accept edits	自動接受改檔，但執行指令還是會問	熟了之後可以換成這個，比較快
Auto	自動執行，背後有安全檢查	這個案例用不到
Bypass permissions	什麼都不問	不要用

> **Plan 模式只套用在目前這個對話**，開新對話要重選。其他模式會記住你在這個資料夾的選擇。

> 舊版本的介面把這幾個模式叫做 Ask permissions、Auto accept edits、Plan mode。名字不同，意思一樣。

三個好用的面板

從 **Views** 選單打開，可以拖到旁邊並排著看：

- **Diff**：Claude 這次改了哪幾行，一行一行對照。**這是你最主要的驗收工具**
- **Browser**：直接預覽你做的網站，不用另外開瀏覽器
- **Terminal**：自己打指令的地方。手冊寫「你自己打」的指令才會用到

Claude 動手之前會問你

在 Manual 模式下，Claude 每次要改檔案或執行指令都會停下來。它會跳出一張卡片等你決定。

看不懂那張卡片在說什麼的時候，不要按允許。 直接在對話框問它：「你剛剛那個動作是要做什麼？為什麼需要？」

接上 GitHub（階段 14 才會用到）

階段 14 要讓 Claude 建一個 GitHub 專案，再把程式碼推上去。它用的是 GitHub 官方的指令列工具 `gh`。這一節把它裝好、登入一次。之後不用再做。

> **名詞**：`gh` 是 GitHub 官方的指令列工具。Claude 用它建專案、推程式碼，不用開網頁。
> 你需要知道到什麼程度：知道它要先登入一次。登入在瀏覽器裡完成，密碼不會經過 Claude。

📋 貼給 AI（裝 gh 並登入）

貼給 AI

你只需要修改：沒有。整段照貼。**第 1 點「我同意之後再裝」不要刪。**

```
幫我安裝 GitHub 的指令列工具 gh，然後登入我的 GitHub 帳號。

1. 先檢查 gh 是不是已經裝了。沒有的話，告訴我你要用什麼方式安裝、
   要執行什麼指令，我同意之後再裝
2. 登入用瀏覽器的方式：gh auth login --web --git-protocol https --hostname github.com
   它會印出一組一次性代碼和一個網址，請把這兩個原樣給我看
3. 我在瀏覽器完成之後，用 gh auth status 確認登入成功
```

你自己做（在瀏覽器完成登入）

1. 打開 Claude 給你的網址（`https://github.com/login/device`）
2. 用你的 GitHub 帳號登入，貼上那組代碼，照畫面指示授權
3. 回到 Claude Code，跟它說「好了」

> **它要你輸入密碼或 token 怎麼辦**：不要在對話框裡輸入。回它「請改用瀏覽器登入的方式」。

✅ 怎麼知道做對了

- [ ] `gh auth status` 顯示已經登入 github.com，後面是你的帳號名稱

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
Mac 上 Claude 說沒有 Homebrew	這台電腦沒有 Homebrew 這個安裝工具	到 cli.github.com 下載安裝檔自己裝，裝完回來從第 2 點繼續
瀏覽器說代碼無效	代碼過期了，或用了上一次的代碼	「代碼過期了，請重新執行一次登入」
Windows 或公司電腦裝不了 `gh`	安裝可能需要 IT 或系統管理員權限	「請說明用 winget 安裝 gh 的方式；如果裝不了，教我改用 GitHub 網站手動建立專案」

這一步失敗可以跳過嗎：可以，只影響階段 14。

課前作業：選十張照片

挑照片的標準

規則是：如果換一個城市（或換一個題材）還完全成立，那張就不要用。

而且這十張要能**排出一個順序**。這個案例用的是一天的時間（04:52 到 00:40）。排不出順序的話，作品就只是十張好看的照片，沒有敘事。

照片從哪裡來

來源	怎麼做	要注意的
自己拍的	直接用	最好的選擇。內容是你的，授權也是你的

來源	怎麼做	要注意的
免費圖庫（例如 Unsplash）	在網站上按下載	練習沒問題。**正式上線前要確認授權條款**

如果你沒有十張照片

不要卡在這裡。三個替代方案，按推薦順序：

1. **換一個你拍得到的題材**：你家附近的一條街、你的通勤路線、一間你常去的店的一天。手機拍就好。
2. **用課程附件裡的十張**：先照著做完一次，理解流程之後再換成自己的。附件裡沒有照片檔，只有每張照片在 Unsplash 上的編號（附件的 `shots/` 是成品截圖，不是素材）。階段 3 建好專案之後，在那個專案裡貼這段（階段 4 步驟一也會再寫一次）：

貼給 AI

```
課程附件解壓縮在桌面的 case-02-tokyo-loop 資料夾。
請讀裡面 src/data/scenes.js 每一筆的 src，從 Unsplash 下載成 public/photos/01.jpg 到 10.jpg。下載之前先告訴我總共多大。
```

這十張是練習用。正式上線前要確認授權。

3. **從免費圖庫湊十張**，但要挑同一個城市、同一種光線邏輯的，不要十張風格各異。

語言的選擇

範例作品的介面文字是日文（`環七、始発前`、`記録をひらく`），因為題材是東京。

你應該用你的題材該用的語言。 如果你做的是台南，就用中文；做倫敦就用英文。介面文字要和照片裡招牌的語言一樣。用錯語言，讀起來就像翻譯過來的。

交作業前的檢查

- [ ] 十張照片排得出一個順序
- [ ] 我可以回答：「這個作品把動畫全部拿掉之後，還剩下什麼？」

最後一題如果答案是「什麼都沒有」，代表題材本身沒有內容、只有效果，建議換題材。**這十張照片和它們的順序要先能講出一件事，動畫才有東西可以襯。**

四張操作卡

這四張卡不是這個案例的內容，是和 AI 一起做事的通用方法。 三個案例裡完全一樣，你之後做自己的專案也用得上。建議印出來，或者截圖存在手機裡。

不用現在背下來。先掃過一遍知道有這四張，遇到下面這些情況再翻回來：

情況	操作卡
要開始一件新的工作，不知道從哪一步下手	**操作卡 1**：七個步驟的順序
Claude 說它要做某件事，你不確定該不該讓它做	**操作卡 2**：哪些可以放手、哪些要你先點頭
想不出指令怎麼寫，或者寫了但它答非所問	**操作卡 3**：一個能用的指令有哪六塊
卡住超過 20 分鐘	**操作卡 4**：卡住的三步處理法
做到後面，Claude 開始違反前面講好的規則	**操作卡 4**：最後一節〈對話太長的時候怎麼辦〉

手冊各階段的 🆘 只處理那一步特有的問題。**手冊沒寫到的情況，回來看這四張。**

操作卡 1｜和 AI 一起做事的七個步驟

```
Step 1  先把需求講清楚
        要做什麼 / 不做什麼 / 有什麼限制 / 怎麼算完成

Step 2  新的、大的改動，先讓 AI 規劃，不要直接改
        什麼時候切到 Plan：
          要做新功能、要大範圍修改、規格還沒談定
        切到 Plan 模式，並明講：
        「先不要修改任何檔案。請告訴我你準備怎麼做、
          會改哪些檔案、每一步的目的，以及可能的風險。」
        手冊寫「這一步用 Manual」的階段，
        是照已經談定的規格或計畫做，直接跳到 Step 4

Step 3  看過計畫，再切回 Manual 讓它執行
        看三件事：有沒有誤解需求 / 有沒有多做不需要的事 /
        有沒有要碰不該碰的檔案或外部資源
        確認完一定要切回 Manual，Plan 模式下它不會動任何檔案
```

```
Step 4  一次只做一小段
        執行 → 看結果 → 驗收 → 確認 → 再往下一步
        不要一次叫 AI 重做整個專案

Step 5  照手冊的驗收條件檢查
        不要用「好像可以」「看起來差不多」
        要用看得到的結果、量得到的數字、跑得出來的測試
        不知道該看哪裡、該怎麼量，就把這一句貼過去：
        「請列出這次改動裡，我在畫面上看不出來、
          但會出錯的地方，以及我要怎麼試。」

Step 6  失敗時不要猜
        把錯誤訊息整段貼給 Claude，再加一句：
        「請先不要修改任何東西。先告訴我最可能的三個原因，
          以及你準備先檢查哪一個。」

Step 7  修完之後重新驗收一次
        「沒有錯誤訊息」不等於「做對了」
```

操作卡 2｜哪些事 AI 可以自己做，哪些要先問你

層級	內容
AI 可以自己做	讀取專案檔案 / 修改本機檔案 / 執行 build（打包） / 執行測試 / 執行檢查 / 分析錯誤 / 建立本機的存檔紀錄（commit）
執行前要你確認	安裝新的工具 / 改動電腦的全域設定 / 大量刪除檔案 / 存取專案資料夾以外的地方
一定要你點頭，它才能做	建立 GitHub 專案 / 第一次把程式碼推上去 / 建立雲端資源 / 部署到公開網址 / 修改正式資料 / 付費 / 發送訊息 / 刪除遠端資源

第三層的原則是：**對外的、要花錢的、不可逆的，AI 不能自己決定。**

重點不是「一定要你親手按」，而是**它動手之前你必須先看到它要做什麼**。這幾件事做下去就收不回來，你要有機會在執行前喊停。正確的說法是：

> 請你先準備好需要的內容與指令，把即將進行的對外操作說明給我聽，我確認之後你再執行。

操作卡 3｜一個能用的指令，有六個構件

手冊上的每一個 Prompt 都是照這六個構件組出來的。**要寫自己的指令時，照同一個順序填。**

#	構件	寫什麼
1	要做什麼	一句話講完
2	不要做什麼	負面清單。這一項最常被漏掉，也最有效
3	限制是什麼	技術、數值、要遵守哪份文件
4	先規劃還是先動手	「先不要改任何檔案，先告訴我你打算怎麼做」
5	怎麼算完成	看得到、量得到、跑得出來的條件
6	哪裡要停下來問我	對外的、要花錢的、要裝東西的

手冊上的 Prompt 怎麼改成你自己的

- `〈…〉` 裡的內容**一定要換掉**
- 其餘的可以照你的專案改寫，但**六個構件盡量都留著**
- **不確定哪裡能動**：手冊上每一段可以貼的 Prompt，上面都有一行「**你只需要修改**：…」。照那一行做就好，沒被點到的部分整段照貼，不用看懂

三個馬上可以用的句型

貼給 AI

> 先不要寫程式碼。用沒有程式背景的人聽得懂的方式，
> 告訴我你打算怎麼做、分成哪幾步、每一步在解決什麼問題。
> 我確認之後你再寫。

貼給 AI

> 只做 **〈這一段〉**，其他的等一下再說。

貼給 AI

有錯先告訴我是什麼，不要直接改。

操作卡 4｜卡住的時候怎麼辦

依序做這三步

第 1 步　照那個階段的「🆘 如果沒有成功」做一次

第 2 步　把下面的 Recovery Prompt 貼給 Claude，一次只處理一個原因

第 3 步　還是不行 → 先跳過

每個階段的 🆘 都會註明「這一步失敗可以跳過嗎」

卡超過 20 分鐘還沒解決，繼續卡下去沒有意義

Recovery Prompt

貼給 AI

我照著教材做到這一步，但沒有得到預期的結果。

我沒有程式背景。

教材說我應該看到：

（貼上手冊裡那一階段的驗收條件）

我實際看到的是：

（描述畫面，或整段貼上錯誤訊息）

請先不要修改任何東西。先告訴我：

1. 最可能是哪三個原因
2. 你準備先檢查哪一個

3. 為什麼是這一個
4. 你需要我提供什麼資訊

一次只處理一個原因。

「整段貼上錯誤訊息」不要偷懶。 真正有用的線索，常常不在最紅、最顯眼的那一行。它在上面幾行不起眼的路徑或版本號裡。

「先不要修改任何東西」也是刻意的。 讓 AI 自己想辦法繞過去，它可能會裝一堆你不知道的東西，或是改掉某個設定。問題會藏得更深。先知道原因，再讓它改。

對話太長的時候怎麼辦

症狀：做到後面，Claude 開始違反前面講好的規則，或是重做已經做過的事。對話越長，前面講過的內容在它眼裡佔的份量越低。

情況	做法
同一件工作還沒做完，對話已經很長	在對話框輸入 `/compact`。它會把前面的對話壓縮成摘要，接著做
這一件做完了，要開始下一件	先確認談好的規則都寫進 `CLAUDE.md`，再輸入 `/clear` 或開一個新對話

`/clear` 之後，Claude 只看得到 `CLAUDE.md` 和專案裡的檔案。談好但沒寫進檔案的決定，換了對話就不見了。接著上一段的 Prompt，例如「照你剛剛列的做」，要和前一段在同一個對話裡做。中間不要 `/clear`。換對話之前先問一句：

貼給 AI

我要開一個新對話繼續做。

這個對話裡談定、但還沒寫進 CLAUDE.md 或任何檔案的決定，請列出來。

我確認之後，請把需要保留的寫進 CLAUDE.md。

不會寫程式，怎麼審查 AI 的產出

下面六個檢查，每一個都不需要看懂程式碼。這個案例的機器檢查只有階段 7 那一條引擎分工規則。多載幾 KB、每一幀多做多少事，沒有機器替你檢查，所以第二項是問它成本。

一、先要它用白話解釋，再讓它寫

在任何一個 Prompt 的最後加上這一段：

貼給 AI

> 先不要寫程式碼。用沒有程式背景的人聽得懂的方式，
>
> 告訴我你打算怎麼做、分成哪幾步、每一步在解決什麼問題。
>
> 我確認之後你再寫。

二、問它成本（這個案例特別重要）

貼給 AI

> 你剛剛用的這個做法，成本是什麼？
>
> - 使用者要多載多少 KB？
> - 每一幀要多做多少事？
> - 在三年前的中階手機上會怎樣？
>
> 如果有更簡單、成本更低的做法，現在告訴我。

套件替你寫掉了程式，但那些計算執行時還是照跑。 你不需要會自己寫，但你需要知道它做的是多大一件事。

三、用眼睛與數字驗收

這個案例的每一個 ✅ 驗收都看得到或量得到。例如畫面上有沒有接縫、停手之後數值是不是真的歸零、四個尺寸有沒有掉幀。

四、問它自己最可能錯在哪

貼給 AI

這次的改動如果會出問題，最可能是哪裡？

請列出三個，並告訴我各自的症狀長什麼樣。

它列出的那三個「症狀」，之後真的出問題時，你可以一項一項對。

五、確認它只改了該改的東西

貼給 AI

你這次改了哪些檔案？每一個各改了什麼、為什麼？

六、紅旗清單：看到這些就停下來問

看到這個	為什麼要警覺
每一幀都在 `new` 一個新物件	每秒產生上百個用完即丟的物件，手機上會週期性卡頓
連續變化的數值被寫成 `gsap.to()`	補間動畫會疊加、互相覆寫（階段 2、7 會解釋）
`importmap` 裡出現 `latest` 或 `^`	版本沒鎖死，哪天對方更新你的網站就壞了
同一個套件從兩個不同網址載入	會載到兩份，外掛會安靜地失效
圖片直接用原檔，沒有壓縮	十張 4000px 的圖，光顯示記憶體就要幾十 MB

你不用自己去程式碼裡翻。每個階段做完，把這張表整張貼給 Claude，請它逐項檢查有沒有中。中了哪一項，就把那一行貼回去問：

貼給 AI

這一行是做什麼的？它的成本是什麼？有沒有更便宜的做法？

這份手冊怎麼用

每個階段最多分成三塊：

```
┌ ⚖️ 動手前先想（有些階段才有）──────
│ 這一步要做的判斷，以及判斷的依據
└──────────────────────────────────

┌ 🛠️ 動手做 ──────────────────────
│ 🧰 這一步的產出
│ 📋 貼給 AI
│ ✅ 怎麼知道做對了
│ 🆘 如果沒有成功
└──────────────────────────────────

┌ 📖 為什麼這樣做 ────────────────
│ 🎯 這一步的重點
│ 🧭 這一步在做什麼
│ 🧩 它在建構流程的位置
│ 💡 那段指令為什麼那樣寫
└──────────────────────────────────
```

先想判斷，再動手，原理在後。 讀完階段開頭那三行提示和〈⚖️ 動手前先想〉就可以動手，做完再回頭讀為什麼。

階段 5 到階段 10，每個階段的 🧰 會多一行「**掛在主迴圈的哪個位置**」。這六個階段是一層一層疊上去的，你要知道自己剛做的東西被誰呼叫。

每個階段開頭還會標一行判斷框架：

```
這一步處理的是：☑ 連續訊號　□ 離散事件　□ 狀態　□ 畫面轉場
```

這四個格子就是階段 2 建立的判斷框架。 十四個階段每一個都標一次，同一個判斷你會做十四次。

Prompt 裡的 `〈…〉` 是要換成你自己內容的地方。

階段 1　先看成品，觀察三件之後會用到的事

📖 讀完約 8 分鐘　🛠️ 動手約 35 分鐘

這一步處理的是：□ 連續訊號　□ 離散事件　□ 狀態　□ 畫面轉場

- **這一步一定要懂**：這個網站不能用滑鼠雙擊打開，要請 Claude 起一個本機伺服器。跑起來之後照三個觀察題各做一次，答案寫下來。再把每個狀態觸發一次，填一張狀態表。
- **這一步可以先不懂**：畫面上那些效果各自是怎麼做出來的。這一步不寫任何程式碼。
- **做完後你要能用自己的話回答**：畫面上哪些東西跟著你的動作在變？至少講出兩個。

🛠️ 動手做

🧰 這一步的產出

- 在 Browser 面板裡跑起來的成品
- 三個觀察題的答案（寫下來，後面會用到）
- 一張填好的狀態表（階段 2 會貼給 Claude）

📋 貼給 AI（1-1：把成品跑起來）

1. 到課程頁面下載 `case-02-tokyo-loop.zip`，解壓縮到桌面
2. 開一個新對話，**Project folder** 選那個資料夾
3. 貼上：

貼給 AI

你只需要修改：沒有。整句照貼。

> 幫我把這個網站跑起來。

4. 從 **Views** 選單打開 **Browser** 面板

> 這個資料夾是**看目標用的**。從階段 3 開始，你會在一個全新的空資料夾裡重做一次。

你自己做（1–2：觀察三件事，填一張狀態表）

看完之後把答案寫下來，後面三個階段會回頭對照。

#	動作	要回答的問題	對應階段
1	先慢慢捲，然後很快地捲	**哪些東西跟著速度變了？**（至少講出兩個）	7
2	從 04:52 一路捲到 00:40，再繼續捲	**有沒有看到「接回開頭」的那一刻？**	8
3	點一張照片，再關掉	**照片是從哪裡飛出來的？**	10

三件事寫完，接著填狀態表。

這個作品有七個狀態。前兩欄已經填好。你要在成品上把每個狀態觸發一次，再填後兩欄。

一般網站的規格寫的是「畫面長什麼樣」。這個案例要寫的是「**做什麼動作，畫面變成什麼樣**」。互動沒辦法用形容詞描述。「要滑順」沒辦法拿去生成，也沒辦法驗收。

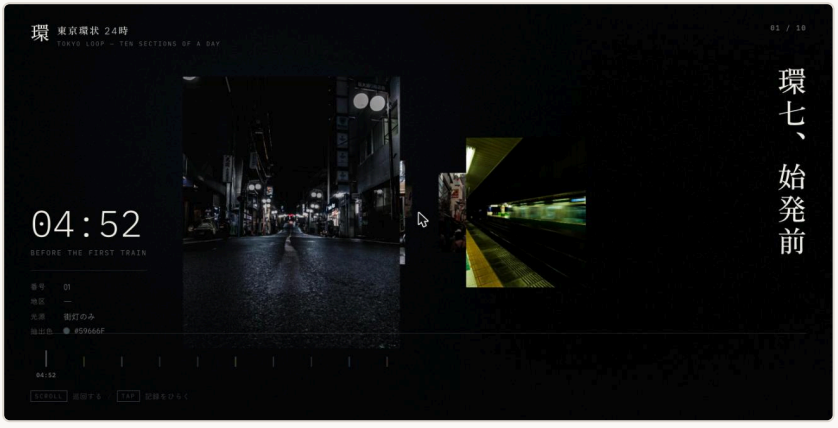
狀態	什麼時候會進到這個狀態	畫面要變成什麼樣	怎麼確認它真的做到了
載入中	剛打開，照片還沒下載完	進度條顯示**實際**載入張數	把網路調慢，進度條會慢慢跑，不會自己衝到 100%
瀏覽	預設狀態，正在捲動	（你填）	（你填）
指到照片上	滑鼠停在某張照片上	（你填）	（你填）
展開	點一下照片	（你填）	（你填）
深色 / 淺色	捲到暗的場景、亮的場景	（你填）	（你填）
靜止	停手 5 秒	（你填）	（你填）
減少動態	系統開了「減少動態效果」	（你填）	（你填）

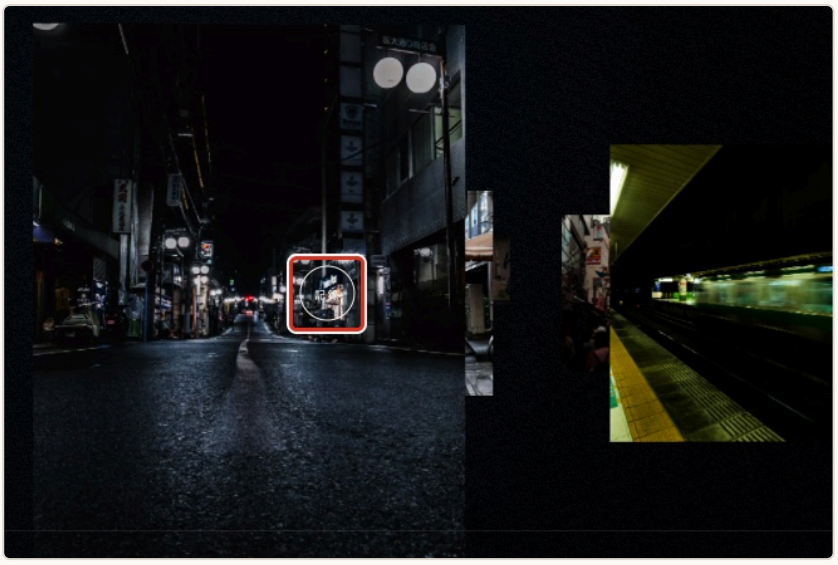
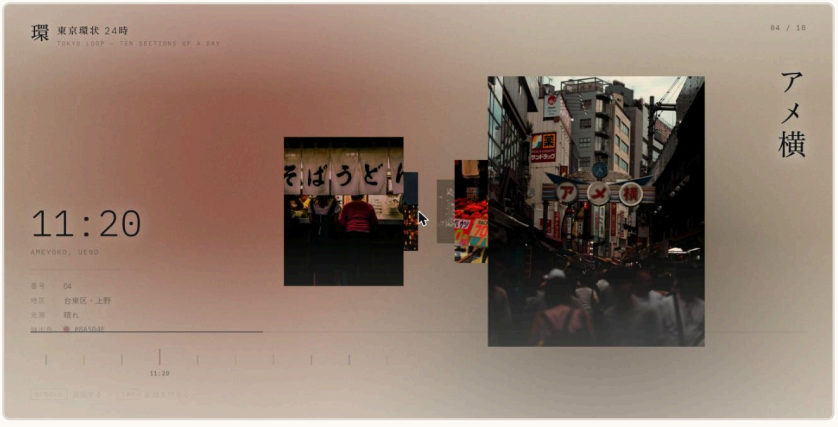
第一列是示範。第三欄寫你看到什麼在變。第四欄寫一個檢查方法，別人照著做就能確認。

第四欄寫不出來的那一列，代表你還沒看清楚它在做什麼。 回成品上再觸發一次。還是寫不出來，就刪掉那一列。

> 「減少動態效果」是作業系統的設定。不知道在哪裡開，問 Claude：「我的作業系統要在哪裡開啟『減少動態效果』？」看完記得關掉。
>
> Windows 11：設定 → 協助工具 → 視覺效果 → 動畫效果（關閉）。 Mac：系統設定 → 輔助使用 → 顯示器 → 減少動態效果。

每個狀態的參考畫面

狀態	參考畫面
載入中	 你要看：中間的進度條和數字 030，代表十張載入了三張。
瀏覽、深色	 你要看：背景是深藍，左邊的時間和右邊的直書標題是淺色字。

狀態	參考畫面
指到照片上	 你要看：這張是局部放大。紅框標出游標變成的「記錄」圓圈。
展開	 你要看：照片停在畫面左邊，右邊是這張照片的時刻、標題和說明文字。
淺色	 你要看：背景變成米色，左邊的時間和右邊的直書標題換成深色字。

狀態	參考畫面
靜止	  你要看：上面是停手前，下面是停手 5 秒後。紅框標出變清楚的操作提示。提示是日文「SCROLL 巡回する / TAP 記録をひらく」，意思是「捲動瀏覽 / 點一下打開記錄」。

「捲得越快，照片邊緣越彎」和「減少動態」是動起來才看得到的差別。截圖看不出來，要在成品上自己試。

✅ 怎麼知道做對了

- [ ] 網站在 Browser 面板裡跑起來了
- [ ] 我寫得出第 1 題的兩個答案
- [ ] 第 2 題我**找不到**接縫（找不到才是對的）
- [ ] 第 3 題我看得出照片是從它原本的位置飛出來的
- [ ] 狀態表至少六列，每一列的第四欄都寫得出來

寫完之後，再對照下面這張參考版本。它是「東京環狀 24 時」實際用的版本。你的第四欄不用和它一字不差，但要做到同一件事：別人照著做，就能確認有沒有做到。

狀態	什麼時候會進到這個狀態	畫面要變成什麼樣	怎麼確認它真的做到了
載入中	照片還沒下載完	進度條顯示**實際**載入張數	把網路調慢（階段 8 會用到，不會調就問 Claude），進度條會慢慢跑，不會自己衝到 100%
瀏覽	預設狀態	捲得越快，畫面越亮、照片邊緣越彎	停手三秒後，所有數值回到 0
指到照片上	滑鼠停在某張照片上	那一張變亮，游標變成一個標記	同一時間只有一張會亮

狀態	什麼時候會進到這個狀態	畫面要變成什麼樣	怎麼確認它真的做到了
展開	點一下照片	照片從**原本的位置**飛到面板左邊	轉場過程中照片不會閃一下、不會跳位
深色	背景亮度低於門檻	整層文字換成淺色	換一組照片資料，反色的時機跟著變
靜止	停手 5 秒	操作提示浮出來	動一下滑鼠，提示又淡回去
減少動態	系統開了「減少動態效果」	持續性的動畫全部停掉，功能還在	開啟後畫面完全靜止，但十張照片仍可瀏覽

🆘 如果沒有成功

> **名詞**：**console** 是瀏覽器裡專門顯示錯誤訊息的那一頁。出錯時紅字會出現在那裡。
> 你需要知道到什麼程度：知道去那裡把紅字整段複製起來貼給 Claude。上面寫什麼你不用看懂。

> **名詞**：**開發者工具**是瀏覽器內建的檢查面板，console 是它的其中一頁。
> 你需要知道到什麼程度：知道怎麼打開，以及 console、Network、Performance 這三頁在哪。其他分頁不用管。
>
> Browser 面板沒有開發者工具。請 Claude 給你本機網址，貼到 Chrome 或 Edge 打開，按 F12（Mac 是 Option + Command + I）。之後手冊每次提到開發者工具，都是用這個方法打開。

你可能會看到	通常是因為	怎麼辦
畫面一片空白	網站還在載入十張照片	等 10 秒。還是空白就看 console 有沒有錯誤
console 出現 CORS 錯誤（瀏覽器擋下跨檔案引用）	用檔案方式打開了	跟 Claude 說「請幫我起一個本機伺服器再打開」
Browser 面板打不開	應用程式版本太舊	Claude → Check for Updates
出現 Python was not found，或跳出 Microsoft Store	Windows 沒有 python3	跟 Claude 說「請改用 npx serve 或 Node 起一個本機伺服器」

> **這一步失敗可以跳過嗎**：可以，但強烈建議想辦法看到。後面的每一個階段都在講這個畫面裡的某一層。

📖 為什麼這樣做

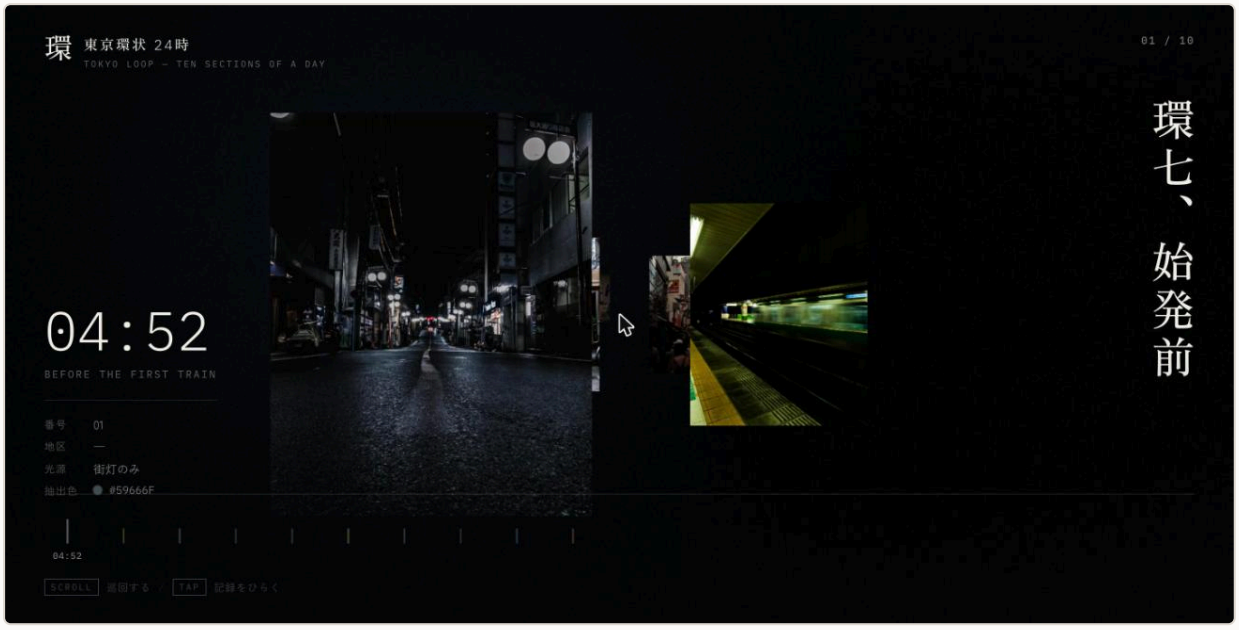
🎯 這一步的重點

看得出畫面上哪些東西在跟著你的動作變，並且說得出至少兩個。

🧭 這一步在做什麼

先把成品跑起來看一次。

這不只是看目標。手冊會請你觀察三件事，它們分別對應階段 7、階段 8、階段 10。先看過一次，後面講原理的時候你腦中才有畫面。



> **你要看的是什麼**：三張照片沿著深度排開，越後面的越小也越暗。它們共用同一台相機，所以透視是一致的。背景是從那張照片算出來的深藍，不是純黑。左邊的時間、右邊的直書標題、下方的時刻索引，全部是一般的網頁文字。不是畫在 3D 場景裡的。這個作品從頭到尾都是這樣：3D 的部分只做 3D 該做的事，其餘交給網頁本來就會的東西。

🧩 它在建構流程的位置

流程開始之前。

狀態表的第四欄怎麼寫

很多人會寫「照片要有視差效果」。問題是：視差多少？跟什麼連動？手放開之後要回到哪裡？

這些沒定義，AI 就會給你一個「看起來有在動」的東西，然後你只能靠感覺調。

改寫成這樣就可以生成、也可以驗收：

> 視差：滑鼠從畫面正中央移到最右邊時，最前面那張照片往左移 `0.28` 個單位。
>
> 越後面的照片移動越少，比例是 `1 ÷ (1 + 深度 × 0.14)`。
>
> 滑鼠移出視窗後，照片回到原位。

差別在哪：上面那句只說了想要什麼，下面這段給了數字。**AI 照著這段做完，你可以拿數字一項一項對。**

💡 那段指令為什麼那樣寫

只有一句「幫我把這個網站跑起來」。但你要知道為什麼不能用滑鼠雙擊打開。

只有一個 HTML 檔的網站，雙擊就能看。這個案例不一樣，檔案之間會互相引用。你用檔案的方式打開，瀏覽器的安全機制會把這些引用擋下來，畫面一片空白。

它必須透過一個本機伺服器提供。Claude 會幫你在自己電腦上起一個。

> **名詞**：**本機伺服器**是一個跑在你電腦上的小程式。它把資料夾裡的檔案用網址的形式送出去，瀏覽器才願意讓這些檔案互相引用。
> 你需要知道到什麼程度：知道它要一直開著，關掉網站就打不開。你不用自己裝，也不用設定。

這是第一個真實的代價：多了一層組裝，就多了一個開啟方式的限制。

階段 2　建立判斷框架：離散事件與連續訊號

📖 讀完約 12 分鐘　🛠️ 動手約 25 分鐘

這一步處理的是：☑ 連續訊號　☑ 離散事件　□ 狀態　□ 畫面轉場

- **這一步一定要懂**：離散（有明確的開始和結束）與連續（每一幀都在變）的分界線。這條線決定一件事交給套件，還是你自己算。
- **這一步可以先不懂**：Three.js、GSAP、shader 各自怎麼運作，WebGL 是什麼。這一步不寫任何程式碼。
- **做完後你要能用自己的話回答**：捲動的阻尼為什麼不能交給補間動畫？做錯會出現什麼症狀？

⚖️ 動手前先想

這一步定下兩件事：哪些效果需要 WebGL，以及每個訊號是離散還是連續。

> **名詞**：WebGL 是瀏覽器裡直接叫顯示卡畫圖的介面。這個作品用的 Three.js 就建在它上面。你需要知道到什麼程度：知道照片邊緣彎曲、背景柔光融合這類逐像素效果要靠它。怎麼呼叫它交給 Three.js。

之後每一個階段開頭的四個格子，都在問你同一件事：這一步在處理的是離散還是連續。

動手之前，先猜一次分界線在哪，寫下來。 做完再到後面〈三、所以分界線在哪〉對照。後面每一個階段都是這條線的應用。

🛠️ 動手做

🧰 這一步的產出

- Claude 對這四個問題的回答
- 狀態表的分工：每一列交給誰
- 你自己版本的分界線說法（寫下來）

📋 貼給 AI（2–1：四個選型問題）

把模式切到 **Plan**（只規劃、不動手），然後：

貼給 AI

你只需要修改：只有第一段的作品描述（如果你做的是別的題材）。**下面四個問題與最後那句「請誠實回答」照貼**，那四個問題才是這一步的產出。

```
我要做一個捲動穿越的 3D 照片藝廊：十張照片沿著深度排開，
捲動時鏡頭穿過它們，背景顏色跟著目前的照片改變，
點擊照片會展開成全螢幕的資訊面板。

先不要寫任何程式碼。請回答四個問題：

1. 這個需求裡，哪些部分非用 WebGL 不可？哪些部分用 CSS 就夠了？
   請逐項說明，不要用一句「用 Three.js 比較好」帶過。
2. 捲動的阻尼與速度，應該用動畫套件的補間動畫，還是自己在
   每一幀的迴圈裡算？兩種做法各自會在什麼情況下出問題？
3. 如果只能引入一個套件，你會選哪一個？
   捨棄另一個之後，哪一部分的品質會下降、下降多少？
4. 這個需求裡有沒有哪一項，其實不是這個題材需要的？

第 4 點請誠實回答。不要為了讓我開心而說它們都有必要。
```

📋 貼給 AI（2-2：把狀態表分給引擎）

接著在同一個對話裡，把你在階段 1 填的狀態表貼上去。

貼給 AI

你只需要修改：只有 `〈…〉`，換成你階段 1 的整張狀態表。**最後那句「不要寫程式碼」照貼。**

這是我在成品上觀察後寫下的狀態表：

〈貼上階段 1 的整張狀態表〉

請一列一列回答：

1. 這個狀態裡會變的東西，是離散（有明確的開始和結束）還是連續（每一幀都在變）？
2. 它該交給 GSAP（Observer、Flip、時間軸）、主迴圈裡自己算，還是 shader？

用一張表回答，每一列寫一句理由。不要寫程式碼。

✅ 怎麼知道做對了

- [ ] 我說得出 Three.js 的價值是「共用相機」和「逐像素運算」，而不是「3D 比較炫」
- [ ] 我說得出 GSAP Observer 解決的是**輸入不一致**，不是「做動畫」（見後面〈二、為什麼需要 GSAP〉）
- [ ] 我舉得出一個例子：什麼情況下把訊號做成補間動畫會出事
- [ ] 我可以用自己的話講完離散與連續的分界線
- [ ] 2-2 的表裡，「瀏覽」被歸成連續、「展開」被歸成離散。不一樣的話，請 Claude 說明理由

🆘 如果沒有成功

你可能會看到：Claude 的回答很籠統，像是「Three.js 適合做 3D 效果」。

怎麼辦：

貼給 AI

你的回答太籠統了。請針對我列出的每一個效果，

一個一個回答「這個效果用 css 做得到嗎？做不到的原因是什麼？」

效果清單：

1. 十張照片沿深度排開、鏡頭穿過去
2. 捲動時照片邊緣彎曲
3. 滑鼠停在某一張上時，只有那一張顏色變濃
4. 背景兩團柔光融合並跟著速度飄動
5. 點一張照片展開成面板

這一步失敗可以跳過嗎：不行。這是這個案例的判斷框架，後面每一步都要用。

📖 為什麼這樣做

🎯 這一步的重點

畫出一條線，而且你要講得出來。線的一邊交給現成的套件，另一邊你自己算。

🧭 這一步在做什麼

這一步不寫程式。你只回答一個問題：這個網站需要哪些工具，為什麼需要。

選錯工具的代價，通常要等到別人的電腦上才看得到。你的筆電不會告訴你。

🧩 它在建構流程的位置

它在流程開始之前，但後面每一個階段都靠它。每個階段開頭那四個格子，就是這一步畫出來的框架。

一、為什麼需要 Three.js

先假設不用它。十張照片就是一般的網頁圖片，用 CSS 做透視。做得出來，但你會撞到三件事。

> **名詞**：CSS 是網頁本來就有的樣式規則，管顏色、位置、大小、轉場這些事。
> 你需要知道到什麼程度：知道它是瀏覽器內建、使用者不用多載任何東西的那一套。這個案例會一直在問「CSS 夠不夠」。

（1）透視是各算各的。

CSS 的透視，每個元素自己算自己的。十張照片不會共用同一台相機。

結果是看起來有點像 3D。視角一變就露餡，因為它們的消失點不在同一個位置。

（2）逐像素的效果做不到。

這個作品有三個效果要一個像素一個像素算：

- 捲動時照片邊緣彎曲
- 滑鼠停在照片上時只有那一張顏色變濃
- 背景兩團柔光互相融合並跟著速度飄開

> **名詞**：跑在顯示卡上、決定每個像素最後顏色的那一小段程式，叫 **shader（著色器）**。
> 你需要知道到什麼程度：知道它吃哪幾個數字就好。這個案例會用到它，但你不會自己寫，也不用讀懂。

CSS 沒有這個能力。

（3）瀏覽器要另外存幾張圖，每一張都佔記憶體。

十張大圖如果各自要求瀏覽器加速，瀏覽器會建立十個獨立的圖層。每一層都要另外存一份。手機的記憶體很快就撐不住。

Three.js 給你的只有三樣東西：

- 一台共用的相機
- 一條你可以自己寫的繪圖流程
- 一個把它們送進顯示卡的橋

Three.js 的文件有幾百頁。這個作品用到十個東西，你可以在 `src/gl/` 三個檔案裡自己數：

用來做什麼	用到的	在哪一階段
把畫面送進顯示卡	`WebGLRenderer`	階段 5
裝東西的容器	`Scene`、`Group`	階段 5、8
兩台相機。一台看照片，一台鋪背景	`PerspectiveCamera`、`OrthographicCamera`	階段 5、6
一塊平面。照片和背景都是它	`PlaneGeometry`、`Mesh`	階段 6、8

用來做什麼	用到的	在哪一階段
自己寫的材質。三個逐像素效果都在這裡	`ShaderMaterial`	階段 6、8
把照片載進顯示卡	`TextureLoader`	階段 8
從滑鼠射一條線出去，逐一比對它打到哪個面	`Raycaster`	階段 10

剩下的都是裝座標和顏色的容器。你不用記名字，看得懂這張表七列各在做什麼就夠了。

> **判斷準則**：如果你的效果只是「移動、縮放、淡入淡出」，**不要**用 Three.js。 CSS 一行能做完的事，換成 3D 只會讓你多載約 750KB。
>
> 該用它的訊號有三個：**要共用同一台相機**、**要逐像素運算**、**物件數量到幾十個以上**。

二、為什麼需要 GSAP

（1）輸入不一致。

滑鼠滾輪回報的「轉了多少」，每台裝置的單位都不一樣。Windows 滑鼠通常是 100 的倍數。Mac 觸控板是帶慣性的小數。Firefox 還有另一個欄位要判斷。觸控裝置根本沒有滾輪事件，你要自己接觸控的按下、移動、放開，再算速度。

自己寫這一段，光是觸控板慣性和手機觸控就會吃掉你好幾天，而且測不完。

GSAP 的 Observer 專門解決這一件事。它把所有輸入抹平成同一組事件。

> **名詞**：**把不同裝置的滑動量換算成同一個單位**，指的就是上面這件事。
> 你需要知道到什麼程度：知道它是一段沒有技術含量、但很花時間的苦工，交出去就好。

（2）位置變換的轉場。

「元素從 A 位置飛到 B 位置」聽起來簡單。但如果 A 和 B 是兩套不同的版面規則，中間沒有可以逐步過渡的共同數值。

GSAP 的 Flip 已經把這件事的陷阱填完了。階段 10 會詳細講。

> **判斷準則**：GSAP 值得用的地方是一次性的、有明確開始和結束的、而且跨瀏覽器差異很大的事。它**不**值得用的地方，是每一幀都在變的連續訊號。

三、所以分界線在哪

離散 ↔ 連續。這就是這個案例的分界線。

- **離散（有明確的開始和結束）**：例如「使用者滾了一格」「面板要打開」
- **連續（每一幀都在變）**：沒有終點。例如「目前的捲動速度」「滑鼠造成的視差位移」

這個數值會怎麼變	交給誰	為什麼
把不同裝置的滑動量換算成同一個單位	GSAP Observer	離散（有明確的開始和結束），而且跨裝置差異大
阻尼、速度、視差、滑鼠停留強度	**自己寫，放在主迴圈裡**	連續（每一幀都在變），沒有起點終點
位置變換的轉場	GSAP Flip	兩套座標系統，一次性
開場、背景壓暗、文字進場	GSAP **時間軸**	有明確的開始、結束與加速曲線
顏色過渡、透明度、彎曲	shader **的變數**	顯示卡端的連續值

> **名詞**：**補間動畫**是「你給起點、終點和時間長度，套件自己算出中間每一幀」。`gsap.to()` 就是在建一個。
> 你需要知道到什麼程度：知道它一建就開始跑，跑完才結束。這個案例要判斷的就是哪些事適合交給它。

> **名詞**：lerp 是「每一幀往目標靠近一點點」，一行就寫得完。這個案例所有的平滑效果都是它做的。
> 你需要知道到什麼程度：知道它每一幀只走剩下距離的一部分，越接近目標走得越慢。

> **名詞**：**主迴圈**是瀏覽器每畫一張新畫面就呼叫你一次的機制。一般螢幕一秒 60 次。階段 5 會把它建起來，之後每個模組都掛上去。
> 你需要知道到什麼程度：知道「每一幀」就是它跑一次。它的正式名稱在最後的名詞白話對照表裡。

兩種畫錯的後果

把每幀訊號做成補間動畫會怎樣？

每次滾輪事件都建一個補間動畫。滾三秒就疊出上百個。它們彼此覆寫、互相打架，畫面延遲又抖動。而且動畫套件每一幀都要走訪所有還活著的補間，等於白白吃掉效能。

把換算滑動量這件事自己寫會怎樣？

你的網站在你的筆電上很順。到同事的 Windows 滑鼠上，一格跳三張。

兩種都不會有錯誤訊息。 選錯工具不會壞掉，只會變爛。

💡 那段指令為什麼那樣寫

剛才那段 Prompt 有四個問題，每一個都是刻意的：

問題	作用
第 1 題	逼它逐項拆。常見的說法是「這個專案用 Three.js」。但正確的說法是「這個專案裡的**這三件事**需要它，其餘用 CSS」
第 2 題	這個案例的判斷框架。讓 Claude 自己講一次，你比較容易接受，而且可以拿它的答案跟上面那張表對照
第 3 題	「只能選一個」會逼出優先順序。你也會看到捨棄一個之後，具體是哪一部分的品質下降
第 4 題	讓 AI 自己指出哪裡是為了效果而效果

🧭 Checkpoint 1（做完階段 2）

口頭回答就好，三題都講得出來就往下走。

1. 用一句話說出離散與連續的分界線。
2. 捲動的阻尼為什麼不能用補間動畫？會出現什麼症狀？
3. 這個作品裡，哪一個效果其實用 CSS 就夠了？

> 第 1 題答不出來，回階段 2 的〈三、所以分界線在哪〉再看一次。

階段 3　建立不用安裝就能跑的專案

📖 讀完約 8 分鐘　🛠️ 動手約 20 分鐘

這一步處理的是：□ 連續訊號　□ 離散事件　□ 狀態　□ 畫面轉場

> - **這一步一定要懂**：版本一定要鎖死。同一個套件只能有一個來源，也就是 importmap 裡 gsap 那三行網址的前半段要完全一樣。以及不裝任何東西的代價是什麼。
> - **這一步可以先不懂**：importmap 的語法、ES Module 是什麼、CORS 的機制。這些照貼給 AI 就好。
> - **做完後你要能用自己的話回答**：什麼情況下，你會把 importmap 換成打包工具？講出兩個。

⚖️ 動手前先想

這個做法可以留到正式環境嗎？ 階段 13 會回頭回答這一題。

importmap 適合的情境是：依賴數量少、不需要移除沒用到的程式碼、希望打開就能跑。

一開始就用打包工具也可以。 這個案例先不用，是為了讓你在裝任何東西之前就先看到畫面會動。到階段 14 你會親手換過去，那時候你說得出換的理由，而不是因為「大家都這樣」。

一旦出現下面任何一項，就該改用打包工具。

> **名詞：打包工具**〈開始之前〉介紹過，這裡再提醒一次。它多了一道「先處理過再上線」的手續，換來的是更小的檔案和更少的連線。
> 你需要知道到什麼程度：知道下面這五種情況出現時，那道手續就值得了。

情況	為什麼 importmap 不夠
要用需要先編譯的語言（例如 TypeScript）	瀏覽器不認得，一定要先編譯
要處理圖片、字體等檔案	importmap 只管程式碼模組
依賴超過五個	每一個都是一次額外的連線
要正式上線	公用服務沒有穩定度保證，掛掉你的網站就開不起來
想減少檔案大小	沒有打包工具就無法移除沒用到的程式碼，Three.js 完整版約 750KB

這個案例最後會撞到其中三項，所以階段 14 會把它換掉。這不是「一開始選錯了」，教學階段用它是對的，撞到上面那三項才需要換。

🛠️ 動手做

🧰 這一步的產出

```
index.html      ← 含 importmap，目前只有一個滿版畫布
styles.css      ← 底色與滿版設定
src/main.js     ← 進入點，畫面上出現一個會轉的方塊
```

畫面上應該看到：黑底，中間一個緩慢旋轉的方塊。

📋 貼給 AI

1. 在桌面建一個空資料夾，名字取 `tokyo-loop`
2. 開一個新對話，**Project folder** 選它
3. 模式選 **Manual**
4. 貼上：

貼給 AI

你只需要修改：沒有。整段照貼。尤其「版本要鎖死」和「指向同一個來源、同一個版本」兩句。刪掉它們就會踩到這一階段講的兩個坑。

```
建立一個純前端專案，不要 npm、不要打包工具、不要框架。條件如下：

- index.html 用 <script type="importmap"> 把 three 和 gsap 對應到 CDN
- 版本要鎖死，不要用 latest，也不要用 ^：three 和 gsap 各給我目前的穩定版
- gsap 的 Observer 和 Flip 兩個外掛，要和 gsap 本體指向同一個來源、同一個版本
- 另外建立 styles.css：黑底、畫布滿版、沒有捲軸
- src/ 底下用 ES Module 分檔，main.js 是進入點
- 這一步只要做到「畫面出現一個會轉的方塊」，先不要做藝廊

另外請回答我一個問題：
為什麼這個專案不能用 file:// 直接打開？請用白話說明。
```

✅ 怎麼知道做對了

- [ ] 打開 **Browser** 面板，畫面上有一個會轉的方塊
- [ ] 我說得出為什麼不能用檔案方式打開
- [ ] importmap 裡沒有 `latest`，也沒有 `^`（打開 `index.html`，最上面那段就是）
- [ ] importmap 裡 gsap 那三行的網址只有最後的外掛名稱不同，網域和版本號完全一樣
- [ ] 我說得出至少兩個「該改用打包工具」的情況（見上面〈⚖️ 動手前先想〉那張表）

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
畫面全黑，沒有方塊	程式有錯	打開 console，整段錯誤訊息貼給 Claude
console 出現 CORS 錯誤	用檔案方式打開了	「請幫我起一個本機伺服器」
網址裡有 `latest`	Claude 沒照做	「請把 importmap 裡的版本全部鎖成具體的版本號」
找不到模組	網址打錯，或那個版本不存在	把 console 的錯誤貼給 Claude

這一步失敗可以跳過嗎：不行。後面所有東西都建在它上面。

📖 為什麼這樣做

🎯 這一步的重點

說得出不用安裝省下了什麼、代價是什麼，以及什麼時候該換掉它。

🧭 這一步在做什麼

建一個不用安裝任何套件就能跑的專案。

> **名詞**：importmap 是寫在 HTML 裡的一小段設定。它告訴瀏覽器「程式碼裡寫的 `import 'three'`，實際上要去哪個網址拿」。
> 你需要知道到什麼程度：知道它是一張網址對照表，改版本就是改那幾行網址。語法你不用背，照貼就好。

這不是這個案例自創的寫法。**Three.js 官方文件教的「不用打包工具」用法就是這一段**，你之後上官網查範例，看到的會是同一組設定。

這一步會踩到兩個坑。兩個都會講清楚。

🧩 它在建構流程的位置

最前面的準備工作。之後每一個檔案都放在這個結構裡。

問題一：用檔案方式打不開

用檔案的方式打開這個網站一定會失敗。網址列開頭是 `file://` 就是這種情況，console 會出現 CORS 錯誤。

> **名詞**：CORS 是瀏覽器的一道安全限制。它不准一份檔案隨便去讀另一份來源不明的檔案。你需要知道到什麼程度：看到 console 出現這四個字母，就代表「你用錯方式打開了」。不是你的程式有錯。

在 Claude Code Desktop 裡這件事不太有感。Claude 會自動幫你起一個本機伺服器，你打開 **Browser** 面板就看得到。

但你要知道它在背後發生了。哪天你把檔案傳給別人，對方雙擊打不開，你才知道要回答什麼。

問題二：同一個套件被載入兩份

如果 `gsap` 和它的外掛對應到不同來源，瀏覽器會載入兩份互不相干的 gsap。例如一個用 A 網站、一個用 B 網站。

你的外掛註冊到第一份，你的動畫跑在第二份上。**外掛會安靜地失效，而且完全沒有錯誤訊息。**

正確寫法是全部指向同一個來源的同一個版本：

```
<script type="importmap">
{
  "imports": {
    "three": "https://esm.sh/three@0.185.1",
    "gsap": "https://esm.sh/gsap@3.15.0",
    "gsap/Observer": "https://esm.sh/gsap@3.15.0/Observer",
    "gsap/Flip": "https://esm.sh/gsap@3.15.0/Flip"
  }
}
</script>
```

> **你要看的是什麼**：gsap 那三行的網址前半段完全一樣，只有最後的外掛名稱不同。這就是「同一個來源、同一個版本」。

> **順帶一提**：GSAP 從 3.13 版開始，所有外掛都免費了，不需要帳號也能用。

💡 那段指令為什麼那樣寫

Prompt 裡「版本要鎖死」那一句不能省。

寫 `latest` 或 `^` 的意思是「幫我拿最新的」。聽起來很好。實際上它代表你的網站會在你沒有改任何東西的情況下自己改變。

哪天對方發布一個有問題的版本，你的作品就壞了，而你根本不知道為什麼。

階段 4　準備照片，並從照片算出背景顏色

📖 讀完約 10 分鐘　🛠️ 動手約 40 分鐘

這一步處理的是：□ 連續訊號　□ 離散事件　☑ 狀態　□ 畫面轉場

- **這一步一定要懂**：顏色是從照片算出來的，不是挑的。還有「你自己電腦上用的工具」和「使用者要載入的東西」是兩回事。
- **這一步可以先不懂**：抽色工具怎麼逐像素統計、彩度怎麼加權。它跑完一次就沒事了。
- **做完後你要能用自己的話回答**：之後要換十張照片，你要改幾個檔案？為什麼是這個數字？

⚖️ 動手前先想

顏色要用挑的還是算的？

你會很想直接說「幫我配十組好看的顏色」。那樣做，配色和照片就沒有關係了。你會得到一組很和諧、但貼到任何照片上都一樣的漸層。

再想一題：顏色從照片算出來之後，早上和深夜的場景要怎麼區分？

先寫下你的答案，做完再到後面〈色相從照片算，明度依時刻調〉對照。

🛠️ 動手做

🧰 這一步的產出

```
public/photos/01.jpg … 10.jpg    ← 你的十張照片
tools/                           ← 抽色工具（不會變成網站的一部分）
src/data/scenes.js               ← 十個場景，每個都帶一組算出來的顏色
```

畫面上還沒有變化，這一步只準備資料。

步驟一：把照片放進去

跟 Claude 說：

貼給 AI

你只需要修改：沒有。整段照貼。

```
幫我在專案裡建一個 public/photos 資料夾。
我等一下會放十張照片進去，檔名是 01.jpg 到 10.jpg，
順序就是它們在網站上出現的順序。
```

然後用 Finder（Windows 是檔案總管）把你的十張照片拖進那個資料夾，照順序改名成 `01.jpg` 到 `10.jpg`。

Windows 要先打開檔案總管的副檔名顯示：檔案總管上方按「檢視」→「顯示」→「副檔名」。否則會改成 `01.jpg.jpg`。 iPhone 的 HEIC 照片請 Claude 轉成 jpg。

用課程附件的十張：附件裡沒有照片檔，只有每張照片在 Unsplash 上的編號。改貼這段，不用自己拖：

貼給 AI

```
課程附件解壓縮在桌面的 case-02-tokyo-loop 資料夾。
請讀裡面 src/data/scenes.js 每一筆的 src，從 Unsplash 下載成 public/photos/01.jpg
到 10.jpg。下載之前先告訴我總共多大。
```

這十張是練習用。正式上線前要確認授權。

步驟二：定資料結構

貼給 AI

你只需要修改：`（你的題材該用的語言）`，以及欄位名稱裡屬於你題材的字（例如「時刻」「場面名」）。**三個規則與最後那句「先給我兩筆」照貼。**

```
現在定資料層。建立 src/data/scenes.js，匯出十個場景的陣列。

每個場景的欄位：
編號、時刻、場面名、羅馬拼音、地區、光源、代表色、說明文字、
```

照片檔名（public/photos/01.jpg 這種）、橫向位置、
三個背景顏色（底色、柔光 1、柔光 2）

三個規則：

1. 三個背景顏色不是我挑的配色，是從照片本身算出來的。
   等一下會寫抽色工具，現在先留空。
2. 地名只在照片本身看得出來時才寫（例如招牌上有寫）。
   看不出來就填「—」，不要猜。
3. 說明文字的寫法是：「照片裡看得到的東西」加上
   「一句只有真的在這個地方待過才寫得出來的觀察」。
   不要寫旅遊文案。

介面文字用 **〈你的題材該用的語言〉**。

先給我兩筆當範例，我確認語氣之後再產其餘八筆。

兩筆讀過、語氣沒問題，就跟它說「語氣可以，請產其餘八筆」。語氣不對就當場說哪裡不對，改到對了再讓它繼續。

步驟三：真的去算顏色

貼給 AI

你只需要修改：沒有。整段照貼，包含縮圖尺寸與三個統計值，不用看懂。

寫一個一次性的抽色工具，放在專案的 tools/ 資料夾。

要求：

- 它是給我在自己電腦上跑的工具，不會變成網站的一部分。
  所以 tools/ 可以有自己的 package.json 和依賴，
  但 index.html 和 src/ 絕對不能引用到它。
- 讀取 public/photos 裡的十張照片
- 每一張縮到 64×80，逐像素統計，輸出三個值：
  1. 平均色
  2. 加權後彩度最高的色（權重要同時考慮出現次數和明度）
  3. 出現次數夠多的最暗色
- 以十六進位色碼印出來，格式要能直接貼進 scenes.js

執行之前先告訴我：你要裝哪一個套件、為什麼需要它、大概多大。

我同意之後你再裝。

跑完之後把結果填進 `scenes.js`，

但明度要依照時刻調整（早上該亮、深夜該暗），色相不要動。

它會提議安裝一個讀圖片用的套件（常見的是 sharp）。這屬於操作卡 2 的第二層：**執行前要你確認**。

同意之前先確認一件事：它裝在 `tools/` 裡，不是裝在專案根目錄。這個區分就是這個案例要教的東西。使用者要載入的東西，和你自己用的工具，是兩回事。

✅ 怎麼知道做對了

- [ ] `scenes.js` 的顏色是跑工具算出來的，不是挑出來的
- [ ] 我可以指著資料說「這個顏色是那張照片裡的某個東西的顏色」

照片和色碼分開看很難對。請 Claude 排在一起：

貼給 AI

做一個暫時的頁面，把十張照片和各自抽出的三個色塊並排給我看，看完刪掉。

- [ ] 早上的場景顏色比較亮，深夜的比較暗（**色相全部來自照片，明度是依時刻調的**）
- [ ] 至少有三筆的「地區」欄位是 `—`（打開 `src/data/scenes.js` 數，範例作品是六筆）
- [ ] 十筆的說明文字裡，沒有任何一句換一個地方還成立
- [ ] 我請 Claude 檢查過：`index.html` 和 `src/` 裡面沒有任何地方引用到 `tools/`

🆘 如果沒有成功

狀況一：套件裝不起來（公司電腦限制、網路擋住）

改用瀏覽器的方式跑，不需要裝任何東西：

貼給 AI

改一個做法：不要裝套件，改成寫一段可以貼到瀏覽器 console 執行的腳本。

請一步一步告訴我：

1. 怎麼在 Chrome 打開 console（我沒有做過）
2. 要貼什麼
3. 跑完之後結果會出現在哪裡、我要複製哪一段

腳本本身的要求和剛才一樣。

注意：如果照片是從別的網站載入的，要加 crossOrigin='anonymous'，

否則讀不出像素。

狀況二：跑出來的顏色全部灰灰的、差不多

通常是因為：只取了平均色。平均色會把所有顏色混成灰。

貼給 AI

現在算出來的顏色太接近了。請確認你有取「加權後彩度最高的色」，

而不是只有平均色。加權要同時考慮出現次數和明度，

只出現幾個像素的鮮豔色不該被選中。

狀況三：只有八張照片

沒關係。改成八個場景，這個作品的每一步都不依賴「剛好十張」。階段 8 的驗收還會請你把數量改成六試試看。

這一步失敗可以跳過嗎：資料層不行。抽色可以先跳過（先手動填十組顏色往下做），但**階段 9 的驗收會做不到**，記得回來補。

📖 為什麼這樣做

🎯 這一步的重點

先把 `scenes.js` 寫完再動畫面，並且看得出「自己挑的配色」和「從照片算出來的顏色」差在哪。

🧭 這一步在做什麼

先把「有哪些照片、每張照片配什麼顏色」定下來。

重點是：**顏色不是用挑的，是從照片本身算出來的。**

🧩 它在建構流程的位置

> 十張照片 ⟶ 【資料層】 ⟶ 主迴圈 ⟶ 背景 / 藝廊 / 介面

這一層是整個作品的來源。 之後所有的顏色、文字、順序，都是從這裡讀出來的。做到後面你會發現：換十張照片＝改這一個檔案。

色相從照片算，明度依時刻調

色相全部從照片算出來，你只依時刻調一次明度。 早上調亮、深夜調暗，色相不動。之後換照片，重跑抽色工具就好，不用重挑顏色。

階段 9 的介面反色，是拿這十組底色的亮度自動切換的。**沒有這一步，反色就只能一個場景一個場景寫死。**

這個判斷可以推廣：換掉素材的時候，手挑的值每一個都要重挑一次。從素材算出來的值不用。

💡 那段指令為什麼那樣寫

資料層的 Prompt 有三個地方可以直接搬到你自己的專案。

「先給我兩筆」：十筆一次產出，語氣如果走偏就要整批重來。先看兩筆，確認語氣對了再繼續。這個習慣在任何長內容生成都適用。

「看不出來就填『—』，不要猜」：原則是寧可留白，也不放不對的內容。AI 很願意幫你補一個聽起來很像的地名，那正是可信度崩掉的地方。範例作品裡有六筆的地區欄位是 `—`，那是刻意的。

「不要寫旅遊文案」：沒有這一句，你會拿到「感受東京的獨特魅力」。

還有一個詞你剛才在 Prompt 裡看過：**加權後彩度最高的色**。白話講就是「這張照片裡出現得夠多、又夠鮮豔的那個顏色」。只有幾個像素的鮮豔色不算，那通常是一盞燈或一個反光點。

階段 5　建立主迴圈，之後每個模組都掛在它上面

📖 讀完約 8 分鐘　🛠️ 動手約 25 分鐘

這一步處理的是：□ 連續訊號　□ 離散事件　☑ 狀態　□ 畫面轉場

- **這一步一定要懂**：後面每一個模組都由主迴圈**每一幀呼叫一次**。另外要懂：解析度上限省下了什麼。
- **這一步可以先不懂**：相機、renderer、怎麼算出照片在螢幕上的位置。這一步你只要確認迴圈真的在跑。
- **做完後你要能用自己的話回答**：畫面上什麼都沒有，為什麼那是正確的結果？

🛠️ 動手做

🧰 這一步的產出

```
src/gl/Engine.js    ← 相機、畫布、縮放、座標翻譯，提供每一幀呼叫的 render()
src/main.js         ← 改寫：建立主迴圈並啟動
```

掛在主迴圈的哪個位置：它自己就是那個位置。

畫面上應該看到：階段 3 的方塊消失，畫面變成一片純色。**看起來什麼都沒有是對的**，模組還沒做。

📋 貼給 AI

貼給 AI

你只需要修改：沒有。整段照貼。**第 4 點（算出照片在螢幕上的位置）現在用不到，但階段 10 只靠它，不要刪。**

```
建立 src/gl/Engine.js，作為整個作品的主迴圈。它負責四件事：

1. 建立 renderer 和一台透視相機。
   相機固定不動（之後做無限循環時，移動的是照片不是相機）。
2. 處理視窗縮放。
3. 提供一個每一幀被呼叫的 render()，之後各個模組會掛進來。
```

4. 提供一個方法：給它一個物件的編號，回傳那個物件投影到
   螢幕上之後的矩形（左、上、寬、高）。這個之後做展開動畫會用到。

三個要求：

- 解析度上限設成 2。請在註解裡說明為什麼要設上限、不設會怎樣。
- 這個檔案只管「怎麼畫」，不管「畫什麼」。場景內容由外面傳進來。
- 用中文註解說明每一段在做什麼。

然後改寫 src/main.js：

- 刪掉階段 3 那個方塊
- 建立主迴圈，啟動每一幀的迴圈
- 迴圈裡先只做一件事：算出這一幀距離上一幀過了多久

做完告訴我：現在畫面上應該看到什麼？

✅ 怎麼知道做對了

- [ ] 階段 3 那個方塊不見了，畫面變成一片純色
- [ ] console 沒有任何錯誤（Browser 面板沒有 console。在 Chrome 打開本機網址再看，方法見階段 1 的〈🆘 如果沒有成功〉）
- [ ] 把瀏覽器視窗拉寬拉窄，畫面不會變形、不會出現黑邊
- [ ] 我說得出解析度上限為什麼要設成 2

問 Claude 這一句確認迴圈真的在跑：

貼給 AI

在每一幀的迴圈裡暫時加一行，每 60 幀在 console 印一次「還活著」。
我確認之後再把它拿掉。

- [ ] console 大約每秒印一次

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
畫面全黑，console 有錯	主迴圈建立失敗	整段錯誤貼給 Claude

你可能會看到	通常是因為	怎麼辦
拉寬視窗畫面變形	縮放處理沒做對	「拉寬視窗時畫面會變形，請檢查 resize 的處理」
console 沒印東西	迴圈沒啟動	「主迴圈好像沒有在跑，請檢查它有沒有被啟動」
方塊還在	沒刪乾淨	「請把階段 3 那個測試用的方塊完全移除」

這一步失敗可以跳過嗎：不行。後面五個階段全部掛在它上面。

📖 為什麼這樣做

🎯 這一步的重點

理解「主迴圈」是什麼，以及為什麼後面每一個模組都要掛在它上面。

🧭 這一步在做什麼

建立整個作品的主迴圈。

這一步之前，畫面上只有階段 3 那個方塊。這一步把方塊換成一個**每秒被呼叫 60 次的迴圈**。之後背景、照片、文字、面板都掛在它上面。

用一句話講：主迴圈負責「現在是第幾幀」，其他模組負責「這一幀我要畫什麼」。

🧩 它在建構流程的位置

```
資料層 ──→ 【主迴圈】 ──┬──→ 背景（階段 6）
                        ├──→ 藝廊（階段 8）
                        ├──→ 介面（階段 9）
                        └──→ 面板（階段 10）
```

這一步很容易被跳過，但跳過的話後面五個階段都會卡住。你會做出一堆檔案，然後發現不知道它們要被誰呼叫。

主迴圈負責的四件事

它負責	白話
相機	決定從哪裡看、看多廣。**十張照片共用這一台**。階段 2 問「哪些部分非用 WebGL 不可」，這是答案之一。背景另有一台，階段 6 再建
畫布與縮放	視窗大小改變時重新算

它負責	白話
每一幀呼叫一次	每畫一張新畫面，就把所有模組叫一遍
算出照片在螢幕上的位置	3D 場景裡的某一張照片，現在對應到螢幕上的哪一塊矩形。**這是階段 10 唯一的橋**

第四件事現在還用不到，但要先做，因為它屬於主迴圈的職責。只有主迴圈同時知道相機和照片在哪裡。

💡 那段指令為什麼那樣寫

有一個數字要特別交代：**畫面解析度的上限**。

高解析度螢幕會回報「一個點要畫 3 個實體像素」。照著畫的話，要畫的像素量是上限設成 2 的**兩倍多**，而肉眼幾乎看不出差別。這是這個案例的第一個效能決定，也是階段 11 那張表的第 1 項。

先回答一題：你的螢幕一個點畫成幾個實體像素？在 Chrome 打開 console，輸入 `devicePixelRatio` 按 Enter，印出來的就是答案。印出 3 的螢幕，這個上限才會生效。印出 1 或 2，這行設定什麼都沒改。

階段 6　做背景：顏色跟著捲動位置變

📖 讀完約 8 分鐘　🛠️ 動手約 25 分鐘

這一步處理的是：☑ 連續訊號　□ 離散事件　□ 狀態　□ 畫面轉場

- **這一步一定要懂**：連續變化的數值每一幀直接覆寫，不做成動畫。顏色來自 `scenes.js`，不寫死在 shader 裡。
- **這一步可以先不懂**：shader 怎麼寫、柔光與顆粒的數學。你只需要知道它吃哪幾個數字。
- **做完後你要能用自己的話回答**：改一組顏色資料，背景立刻跟著變，這件事證明了什麼？

⚖️ 動手前先想

離散還是連續：背景顏色跟著捲動位置變。它有明確的開始和結束嗎？

接著想：它該做成補間動畫，還是每一幀直接覆寫 uniform？

> **名詞**：uniform 是從 JavaScript 傳給 shader 的數字，例如背景的底色、時間、速度。
> 你需要知道到什麼程度：知道它每一幀可以直接覆寫就夠了。

先寫下你的答案，做完再到後面〈背景顏色為什麼不用補間動畫〉對照。

🛠️ 動手做

🧰 這一步的產出

```
src/gl/Background.js    ← 全螢幕背景，顏色跟著進度變
```

掛在主迴圈的哪個位置：主迴圈每一幀先畫它，再畫其他東西。

畫面上應該看到：一片會緩慢流動的底色，上面有兩團柔光。

📋 貼給 AI

貼給 AI

你只需要修改：沒有。整段照貼。中間那三個「請特別注意」各對應一個真的會發生的問題。

```
建立 src/gl/Background.js，掛到主迴圈上。要求：

- 用一個獨立的場景畫全螢幕背景，在主場景之前繪製
- shader 的內容：底色 + 兩團柔光 + 顆粒
- 傳進去的數字至少要有：底色、兩個柔光色、時間、速度、長寬比
- 提供一個 update(進度, 速度, 時間)，
  依照「目前在哪兩個場景之間」即時過渡三個顏色

三個要求請特別注意：
1. 長寬比要修正，否則圓形的柔光會被拉成橢圓
2. 顏色過渡不要每一幀都建立新的顏色物件，要用事先建好的暫存物件。
   請在註解裡說明為什麼
3. 加顆粒的目的是蓋掉純色漸層在暗處會出現的色帶，不是為了好看。
   請把這個理由寫在註解裡

目前還沒有捲動訊號，所以先在 main.js 裡用時間當假的進度，
讓背景自己慢慢在十個場景之間循環，這樣我才看得出它有沒有在動。

用中文註解說明每一段在做什麼。
```

✅ 怎麼知道做對了

- [ ] 畫面上有一片會緩慢變化的底色和兩團柔光
- [ ] 改 `scenes.js` 裡任何一組顏色，背景立刻跟著變，完全沒有碰 shader
- [ ] 把視窗拉成很寬的橫條，柔光還是圓的（不是橢圓）
- [ ] 我說得出為什麼背景顏色不用補間動畫

第二項才是這個階段要驗的東西。不會自己改檔案就跟 Claude 說「把第 3 個場景的底色改成紅色，我看完再改回來」。**它證明了資料和呈現是分開的**，之後換十張照片，你只要改資料。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
畫面全黑	背景沒有被畫出來	「背景好像沒有被繪製，請檢查繪製順序和 autoClear 的設定」
柔光是橢圓形	長寬比沒修正	「柔光在寬螢幕上被拉成橢圓，請修正長寬比」
顏色不會變	假訊號沒接上	「背景顏色都不會變，請確認 update 有被每一幀呼叫」
暗處有一圈一圈的階梯	顆粒沒加或太弱	「暗處出現色帶，請把顆粒的強度調高一點」
改資料背景沒變	顏色被寫死在 shader 裡了	「顏色應該完全來自 scenes.js，請把 shader 裡寫死的顏色移除」

這一步失敗可以跳過嗎：不建議。這是第一個掛在主迴圈上的模組，這一步順了，後面三個都是同樣的模式。

📖 為什麼這樣做

🎯 這一步的重點

第一次接觸 shader，並理解傳給它的數字是「每一幀直接覆寫」，不是「做成動畫」。

🧭 這一步在做什麼

做背景。它會跟著你捲到哪裡而改變顏色。

這是第一次碰到「跑在顯示卡上的程式」，但你只需要看懂它吃哪幾個數字。

🧩 它在建構流程的位置

> 資料層 ⟶ 主迴圈 ⟶ 【背景】

掛在主迴圈的哪個位置：在所有東西之前繪製，當作底。

現在還沒有捲動訊號（那是階段 7），所以這一步先用時間當假訊號讓它動起來。先讓背景會動，你才看得出階段 7 的訊號接上去之後對不對。

背景顏色為什麼不用補間動畫

背景顏色的過渡是連續（每一幀都在變）的訊號。所以它每一幀直接覆寫 uniform，不建補間動畫。

理由和階段 2 一樣：捲動沒有終點，做成補間動畫會疊出上百個互相打架的東西。

兩個名詞

名詞	白話
shader	一段跑在顯示卡上的小程式，決定每個像素的顏色
色帶	純色漸層在暗處出現的一圈一圈階梯。因為每個色階只有 256 級不夠細

💡 那段指令為什麼那樣寫

Prompt 裡有三個要求，每一個都對應一個真實會發生的問題。

「不要每一幀 new 一個顏色物件」，這是這個案例第一個效能教學點。每秒 60 幀乘上每幀三個物件，等於每秒產生 180 個用完即丟的物件。這些物件會累積在記憶體裡，等到瀏覽器自動清理時就會卡一下。

> 這種卡頓在筆電上不容易看出來。手機記憶體小，清理更頻繁，卡一下的次數會變多。這就是階段 11 要處理的那一類問題。

畫面上看不出來，要怎麼知道程式裡有沒有這種寫法？量的方法在階段 11：記憶體用量呈鋸齒狀。查的方法是直接請 Claude 列出來：

> 貼給 AI
>
> 請列出所有每一幀都會執行的程式裡，用 `new` 建立物件的地方。
>
> 每一處標出檔名和行號，先不要改。

「長寬比要修正」：不修正的話，圓形的柔光在寬螢幕上會被拉成橢圓。

「顆粒是為了蓋掉色帶」要寫進註解。 加顆粒不是為了好看，是為了打散暗處的階梯。不寫理由，下一個人會把這幾行當成隨手加的效果。改版時它第一個被刪，色帶就回來了。

Prompt 沒寫、但背景一定要有的一個設定。 先想一題：背景這塊平面是 shader 直接鋪到螢幕上的，沒有經過相機。 Three.js 判斷「它在不在畫面內」時，看的卻是它在 3D 場景裡的位置。這兩

件事對得上嗎？對不上的症狀是背景整片消失，console 沒有錯誤。這是階段 11 那張表的第 6 項。做完背景請問一次：

貼給 AI

背景那塊平面有沒有關掉「畫面外就不畫」的檢查（`frustumCulled`）？

沒有的話請關掉，並在註解寫原因。

🧭 Checkpoint 2（做完階段 6）

第一個模組已經掛到主迴圈上了。這三題確認你知道自己剛剛疊了什麼。

1. 你改了 `scenes.js` 的一組顏色，背景立刻變了。這證明了什麼？
2. 主迴圈負責什麼？背景模組負責什麼？兩者的分工是什麼？
3. 現在背景是靠什麼在動？（提示：真正的訊號還沒接上）

> 第 3 題答不出來，回階段 6〈🧩 它在建構流程的位置〉那段「先用時間當假訊號」再看一次。下一階段就是要把假訊號換成真的。

階段 7　做捲動訊號：哪一半交給套件、哪一半自己寫

📖 讀完約 15 分鐘　🛠️ 動手約 50 分鐘

這一步處理的是：☑ 連續訊號　☑ 離散事件　□ 狀態　□ 畫面轉場

- **這一步一定要懂**：同一個檔案裡的兩半：接收輸入交給套件、阻尼與速度自己算。以及為什麼一定要有歸零門檻和速度上限。
- **這一步可以先不懂**：套件怎麼把各種裝置的輸入抹平、會循環的位置刻度要怎麼取餘數。
- **做完後你要能用自己的話回答**：指著這個檔案說出哪幾行是套件做的、哪幾行是你做的？

⚖️ 動手前先想

離散還是連續，這裡第一次真的動手分。

同一個檔案裡，兩件事要分開：

這一半	交給誰	為什麼
把不同裝置的滑動量換算成同一個單位	GSAP Observer	離散（有明確的開始和結束），而且跨裝置差異大
阻尼、速度、視差、靜止時間	**自己寫，放在主迴圈裡**	連續（每一幀都在變），沒有起點終點

下面那段 Prompt 最後要求在檔案最上方寫中文註解，說明為什麼這樣分。這個要求的作用是：

> 三個月後回來改這份程式碼的人（可能是你自己，也可能是下一次的 AI），會先讀到那段註解，然後才動手。

🛠️ 動手做

🧰 這一步的產出

```
src/input/Scroll.js        ← 整個作品唯一的捲動訊號來源
CLAUDE.md                  ← 引擎分工規則（7-3）
scripts/engine-lint.mjs    ← 檢查那條規則的腳本（7-4）
.claude/settings.json      ← 把檢查掛上去的設定（7-4）
```

掛在主迴圈的哪個位置：每一幀最先更新，輸出的數值餵給其他所有模組。

畫面上應該看到：捲動時背景顏色跟著變（不再是自己慢慢跑）。

📋 貼給 AI（7–1：訊號來源）

貼給 AI

你只需要修改：沒有。整段照貼。**最後那段「最後請在檔案最上方用中文註解寫清楚」是這一步的重點，不要刪。**

建立 `src/input/Scroll.js`。這個類別是整個作品唯一的捲動訊號來源。

用 `GSAP Observer` 負責接收輸入：
- 同時接滾輪和觸控
- 要阻止瀏覽器的原生捲動
- 收到事件時只做一件事：把移動量累加到「目標位置」

其餘全部自己在 `update()` 裡算，不要用任何補間動畫：
- 目前位置用「每一幀往目標靠近一點點」的方式追目標（這是阻尼）
- 速度是目前位置每一幀的變化量，再平滑一次，並限制在上下限內
- 低於門檻就直接歸零，否則永遠停不下來
- 提供一個 `0` 到 `1` 的正規化速度，給 `shader` 用
- 提供拖曳量，讓照片跟著手勢方向偏一點
- 提供「已經靜止幾秒」

位置不要設上下限。這是無限捲動。

最後請在檔案最上方用中文註解寫清楚：
為什麼接收輸入交給套件、而阻尼和速度留在每一幀的迴圈裡自己寫。

這段註解是寫給之後讀這份程式碼的人看的，請用白話寫，

不要只寫「為了效能」。

然後把背景那邊的假訊號換成真的捲動訊號。

📋 貼給 AI（7–2：跳到某個場景）

貼給 AI

你只需要修改：沒有。整段照貼。

加一個方法，讓外部可以直接跳到某個場景。

因為這是無限捲動，位置是在繞圈的座標裡，

直接設定「目標 = 編號 × 間距」會出事：

從第 10 個跳回第 1 個，畫面會整整倒帶九張。

請算出繞圈座標裡的最短路徑再跳，並在註解裡說明這個陷阱。

順帶提醒：JavaScript 的取餘數對負數會回傳負值，

請確認你的算法有處理這件事。

📋 貼給 AI（7–3：把分界線寫成規則）

7–1 的分界線現在只存在這個對話和 `Scroll.js` 的註解裡。階段 8 和階段 10 還要再寫兩個模組，到時候 Claude 會用最順手的寫法，而最順手的寫法就是 `gsap.to`。這一步把分界線寫進每次對話都會先讀的 `CLAUDE.md`（〈開始之前〉解釋過）。

貼給 AI

你只需要修改：沒有。整段照貼。**第 3 點「寫清楚理由」不要刪。**

在專案根目錄的 CLAUDE.md（還沒有就建立）加上這個專案的引擎分工規則。

已經有的內容不要動：

1. 一張表，列出五種動態值各自交給誰、放在哪個檔案：

   輸入正規化 → GSAP Observer → src/input/Scroll.js

   阻尼、速度、拖曳量、視差 → 主迴圈裡手寫 lerp →

```
src/input/Scroll.js、src/gl/Gallery.js、
      src/main.js（指標視差）
   佈局轉場 → GSAP Flip → src/ui/Detail.js
   狀態切換（壓暗、進場、開場）→ GSAP Timeline → src/main.js
   顏色插值、透明度、彎曲 → shader uniform → src/gl/Background.js、
src/gl/Gallery.js
   還沒建立的檔案也寫進去，後面的階段照這張表放。

2. 一條規則：src/gl/ 和 src/input/ 裡不准呼叫 GSAP 補間
   （gsap.to、gsap.from、gsap.fromTo、gsap.timeline、gsap.quickTo、Flip.from）。
   Observer.create 和 gsap.registerPlugin 不算，那是接收輸入。

3. 用白話寫清楚這條規則的理由：這兩個資料夾的值每一幀都在變，
   用補間會發生什麼事，畫面上看得到什麼症狀。

用繁體中文寫。
```

📋 貼給 AI（7–4：做成自動檢查）

> **名詞**：**hook** 是在 Claude 做某個動作前後，由 Claude Code 自動執行的檢查。
> 你需要知道到什麼程度：知道它由工具執行，Claude 沒辦法忽略它。違反了就把訊息送回去，要求 Claude 自己修正。

貼給 AI

你只需要修改：沒有。整段照貼。**最後那行指令一個字都不要動**，引號是在處理資料夾路徑有空格的情況。

```
建立 scripts/engine-lint.mjs，在 Claude 每次改完檔案後，自動檢查 CLAUDE.md 裡那條
規則。

行為規格：
- 從標準輸入讀 Claude Code 傳來的資料（JSON 格式），取出被修改的檔案路徑
- 只檢查 src/gl/ 和 src/input/ 底下的 .js 檔，其他直接放行
- 註解裡出現的 gsap.to 不算違規
- Windows 的路徑是反斜線，也要判斷得出來
- 違規時把檔名、行號和那一行印到標準錯誤（stderr），結束代碼是 2。
  印到一般輸出的話，Claude 收不到這個訊息
```

```
- 加一個 --all 參數，可以手動掃整個 src/，沒有違規時印出「全部通過」

錯誤訊息用繁體中文，說明為什麼不行，以及該改成什麼寫法。
這個訊息是要給 Claude 看的，它需要知道理由才能修對。

然後建立 .claude/settings.json，把它掛在 Write 和 Edit 之後執行，指令是：
  node "$CLAUDE_PROJECT_DIR/scripts/engine-lint.mjs"
```

📋 貼給 AI（7-5：證明它真的會擋）

貼之前先在對話框輸入一次 `/hooks`，或開一個新對話，讓剛建的設定生效。

這個檢查在 Claude **改完檔案之後**才跑。被退回的時候，違規的那一行已經寫進檔案了，要等 Claude 自己改回來才算完成。

貼給 AI

你只需要修改：沒有。整段照貼。這一段的重點是親眼看到「退回 → 收到訊息 → 自己修好」。

```
現在測試檢查是不是真的有效。

請故意把 src/input/Scroll.js 的 update() 裡，
「目前位置追目標」那一行改成用 gsap.to 追。

我要看檢查攔截你、你收到錯誤訊息、然後你自己改回 lerp 的完整過程。
最後跑一次 node scripts/engine-lint.mjs --all，給我看結果。
```

✅ 怎麼知道做對了

- [ ] 捲動時背景顏色跟著變
- [ ] 觸控板、滑鼠滾輪、手機觸控，三種輸入的手感一致（**至少測兩種**。只有觸控板的話，先測觸控板，手機留到階段 12）
- [ ] 我可以指著檔案說出哪幾行是套件的工作、哪幾行是我的工作。依據是檔案最上方的中文註解
- [ ] 檔案最上方那段註解，我讀得懂

請 Claude 幫你驗證歸零：

貼給 AI

在 `console` 每秒印一次目前的速度值。

我要停手三秒之後，看到它真的是 `0`，不是 `0.0001`。

確認完再把這行拿掉。

- [ ] 停手三秒後，速度真的是 `0`
- [ ] `CLAUDE.md` 裡有引擎分工表和那條規則，規則旁邊寫了理由
- [ ] 我親眼看到 Claude 改完之後被檢查退回，然後自己改回 lerp（7-5）
- [ ] `--all` 最後印出「全部通過」

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
捲動沒反應	Observer 沒啟動	整段 console 錯誤貼給 Claude
頁面還會上下捲	沒有阻止原生捲動	「請確認 Observer 有阻止瀏覽器的原生捲動」
停手後速度停在 0.0001	沒有歸零門檻	「速度停不下來，請加上歸零門檻」
滾一下跳很多張	沒有速度上限，或阻尼太弱	「請加上速度上限，並把阻尼調得再慢一點」
手感很鈍	阻尼太強	「阻尼太強了，請把每一幀靠近目標的比例調高一點」
7-5 改成 `gsap.to` 之後沒有被擋	對話開始時 `.claude/` 還不存在，設定沒被讀到	在對話框輸入一次 `/hooks`，或開一個新對話再試
沒有違規的檔案也被擋	檢查範圍太大	「engine-lint 只該檢查 src/gl/ 和 src/input/，請把範圍收窄」

這一步失敗可以跳過嗎：7-1、7-2 不行。這是整個作品的訊號來源，後面三個模組都吃它。 7-3 到 7-5 可以跳過，但階段 8 和階段 10 把補間寫錯位置時，沒有東西會擋。

📖 為什麼這樣做

🎯 這一步的重點

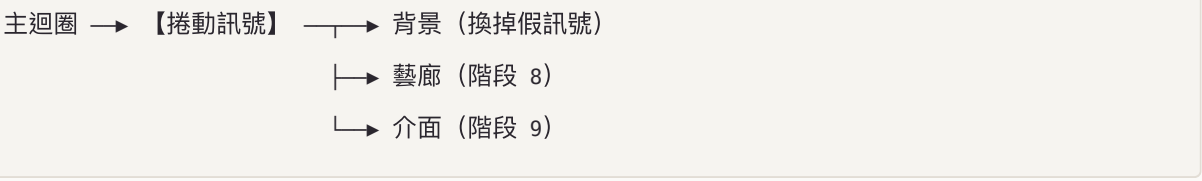
在同一個檔案裡，親手把「該交給套件的」和「該自己寫的」分開。

🧭 這一步在做什麼

做出整個作品**唯一的捲動訊號來源**。

界線畫錯有兩種後果：捲動延遲又抖動，或是換一台裝置就一格跳三張。兩種都不會有錯誤訊息。

🧩 它在建構流程的位置



掛在主迴圈的哪個位置：每一幀最先更新，因為其他所有模組都吃它的輸出。

三個一定要理解的細節

（1）接管捲動之後，原生的捲動就沒了。

這代表整個頁面的捲動行為變成你的責任，也代表**無障礙的責任變成你的**。階段 12 會補上鍵盤操作。

（2）為什麼一定要有「歸零門檻」。

平滑的做法是「每一幀往目標靠近一點點」，數學上永遠到不了目標值。不歸零的話，速度會停在 0.0001 這種數字。畫面會一直微微發亮，顯示卡每一幀都在算一個看不見的差異。

（3）為什麼一定要有「速度上限」。

使用者可以瘋狂滾兩秒。沒有上限的話，照片會扭曲到看不出原樣。換算成 0 到 1 的速度也會超過 1，shader 的計算跟著失準。

💡 那段指令為什麼那樣寫

第二個 Prompt（跳到某個場景）在處理一個很容易被忽略的問題。

因為這是無限捲動，位置是在一把**會循環的位置刻度**上，Prompt 裡叫它「繞圈的座標」。直接把目標設成「第 10 個 × 間距」會出事：從第 10 個跳回第 1 個，畫面會整整倒帶九張。

正確做法是算出這把刻度上的最短路徑再跳。**這是「用了無限捲動就要付的第二個代價」**。所有跟位置有關的計算，都要改成會循環的算法。

7–1 的 Prompt 裡還出現了一個詞，講的是**換算成 0 到 1 的速度**。shader 只吃範圍固定的數字，超過 1 它的計算就失準，這也是上面那個速度上限存在的理由。

為什麼判斷完還要寫成檢查

7–3 到 7–5 的做法是先寫成 `CLAUDE.md` 的規則，再做成自動檢查。規則的內容是「離散還是連續」的結論。

這條檢查只看檔案位置，不分析程式邏輯。把每一幀更新的程式寫進 `src/main.js`，它就抓不到。所以 7–3 那張表的「放在哪個檔案」一欄，和規則本身一樣要遵守。

階段 8　讓照片無限循環，而且看不出接縫

📖 讀完約 10 分鐘　🛠️ 動手約 35 分鐘

這一步處理的是：☑ 連續訊號　□ 離散事件　☑ 狀態　□ 畫面轉場

- **這一步一定要懂**：回收的距離一定要遠在淡出完成的距離之外。「無限」是重算既有物件的位置，不是準備更多物件。
- **這一步可以先不懂**：深度緩衝、繪製順序這兩個設定的技術細節。照著 8–2 貼就好。
- **做完後你要能用自己的話回答**：為什麼看不到接縫？請用「兩個數字的關係」回答。

🛠️ 動手做

🧰 這一步的產出

```
src/gl/Gallery.js    ← 十個平面沿深度排開，會無限循環
```

掛在主迴圈的哪個位置：背景之後、介面之前。

畫面上應該看到：你的十張照片沿深度排開，捲動時鏡頭穿過它們。後面 🧭 有一張成品截圖可以對照。

📋 貼給 AI（8–1：無限循環）

貼給 AI

你只需要修改：只有「十個平面」的數量（如果你的照片不是十張）。其餘照貼。

```
建立 src/gl/Gallery.js，掛到主迴圈上。
十個平面沿著深度方向排開，無限循環。

作法：相機不動，每一幀用取餘數把每個平面的位置折回可見範圍。
不要複製物件，不要用第二組，就是這十個。

每個平面疊四層微動態，全部共用階段 7 那一組訊號：
1. 跟著滑鼠移動，越深的平面移動越少
2. 跟著捲動方向拖曳
```

3．速度越快，傾斜和縮放越明顯

4．由距離算出來的透明度

透明度必須滿足一個條件：

回收的距離要遠在淡出完成的距離之外，

否則使用者會看到照片憑空出現。

請把這兩個數字的關係寫在註解裡，並說明為什麼不能亂調。

淡出完成的距離要從照片張數算出來，不要寫死。

張數變少時回收會提早，淡出也要跟著提早。

用中文註解說明每一段在做什麼。

📋 貼給 AI（8–2：半透明三件套）

貼給 AI

你只需要修改：沒有。整段照貼。

現在照片會互相穿插。請處理半透明物件的繪製順序：

- 材質設成半透明，並關掉深度寫入
- 每一幀依照距離指定繪製順序，近的畫在上面

請順便用白話解釋兩件事：

1．為什麼深度寫入要關掉

2．關掉之後為什麼還需要自己指定繪製順序

📋 貼給 AI（8–3：載入畫面）

貼給 AI

你只需要修改：只有照片張數。第 2 點（不要用假的進度條）照貼。

十張照片載入需要時間，現在這段時間畫面是空的。加一個開場畫面：

1. 蓋住整個畫面，顯示作品名稱和一個進度條
2. 進度條要顯示「實際載入完成的張數 ÷ 總張數」，
   不要用假的動畫，也不要自己衝到 100%
3. 全部載入完成之後淡出

第 2 點請特別注意：我要能夠把網路調慢，然後看到進度條真的慢慢跑。

網路要在 Chrome 的開發者工具裡調慢（打開方法見階段 1 的〈🆘 如果沒有成功〉）。不會調就問 Claude：「請一步一步告訴我怎麼在 Chrome 開發者工具把網路調慢。」

✅ 怎麼知道做對了

- [ ] 從頭捲到尾再捲回來，**看不到任何照片「出現」的瞬間**
- [ ] 照片交錯的地方沒有方形破洞
- [ ] 我說得出為什麼回收距離一定要大於淡出距離
- [ ] 進度條會實際反映載入張數

動手做一次這個實驗（這比讀十遍有用）：

貼給 AI

請暫時把淡出結束的距離改成比回收距離還大的值，讓我看看會發生什麼。
如果程式裡有「淡出一定要在回收之前完成」的保護（例如兩個距離取比較小的那個），這次也一起暫時拿掉。
我看完之後你再全部改回來。

- [ ] 我親眼看到照片啪一下出現在遠處
- [ ] 改回來之後又看不到了

還有一個測試資料層是否乾淨的方法：

貼給 AI

請暫時把場景數量從十個改成六個，讓我確認不用改其他程式碼也能正常運作。

確認完再改回來。

- [ ] 改成六個之後照樣正常循環，而且一樣看不到照片憑空出現

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
照片憑空出現	淡出比回收晚	「照片會憑空出現，請確認淡出在回收之前就完成」
交錯處有方形破洞	深度寫入沒關	「照片交錯的地方有方形破洞」
只看得到一張照片	橫向位置太集中，全被最前面那張擋住	「請把各張照片的橫向間距拉開」
照片閃爍	繪製順序每幀都在跳	「照片在閃爍，請自己明確指定繪製順序」
進度條直接跳到 100%	用了假動畫	「進度條要反映實際載入張數，不要用假的動畫」

這一步失敗可以跳過嗎：不行。這是作品的主體。

📖 為什麼這樣做

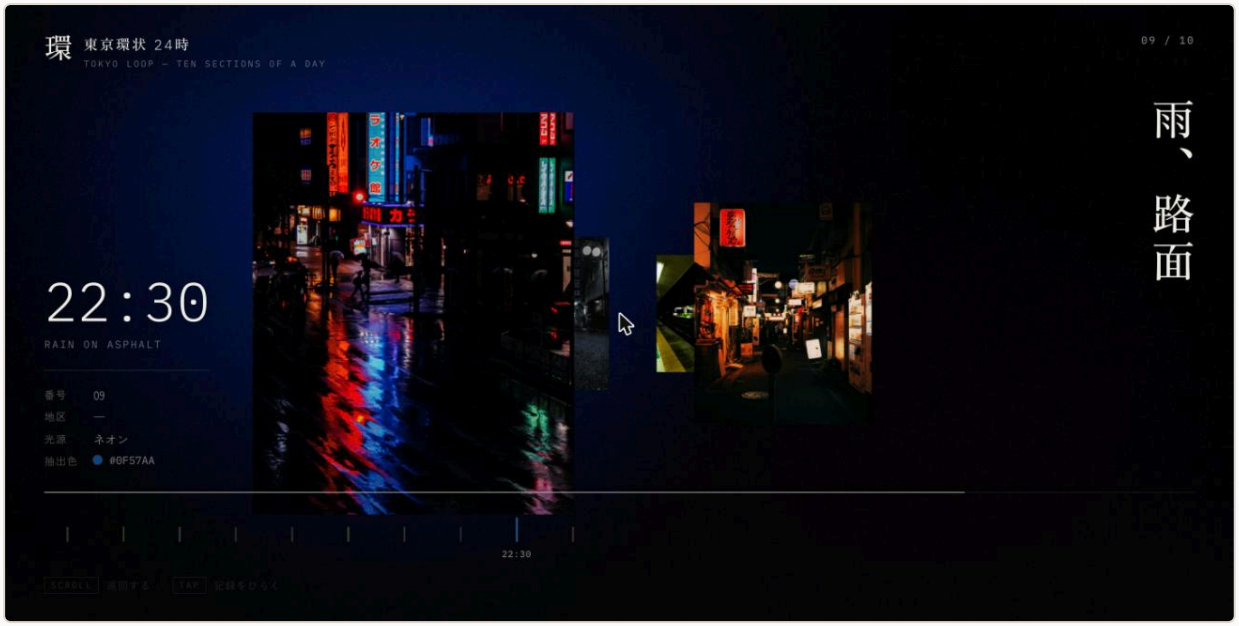
🎯 這一步的重點

「無限」在 3D 裡不是準備更多物件。是把既有的物件重新算一次位置。

🧭 這一步在做什麼

讓照片一直捲下去。捲到最後會接回開頭，而且你看不出接縫。

作法一句話講完：照片被相機穿過之後，位置就搬到最後面那一張的後方，重新排一次。全程就是這十張在繞。



> **你要看的是什麼**：最前面那張最大最亮，後面兩張逐漸變小變暗。 **最後面那張已經淡到快看不見了。** 這不是美術效果，是這一步的技術條件：照片必須在被搬走之前就完全透明，否則你會看到它憑空出現。

🧩 它在建構流程的位置

```
捲動訊號 ──→ 【十張照片無限循環】 ──┬──→ 介面（階段 9）
                                   └──→ 面板（階段 10）
```

掛在主迴圈的哪個位置：每一幀在背景之後繪製，吃捲動訊號的輸出。

這一步只要記住一句話

> **回收的距離，要遠在淡出完成的距離之外。**

在這個作品裡，十張照片時，照片在距離 24.6 的地方被回收，淡出在 15.5 就完成了。中間留了 9.1 個單位的安全區。

接縫看不到，靠的就是這 9.1 個單位。 沒有它，你要寫一套排程去藏接縫。有了它，取餘數三行就夠了。

半透明三件套

照片會互相穿插，所以要處理繪製順序。這裡有三個設定必須一起做，後面就叫它**半透明三件套**。材質設成半透明、關掉深度寫入、自己指定繪製順序。少做任何一個都會出事。

（1）為什麼要關掉「深度寫入」。

> **名詞：深度寫入**就是「誰擋住誰的紀錄」。顯示卡把「這個像素目前最近的物件有多遠」記下來，用來讓後面的東西不要蓋住前面的。
> 你需要知道到什麼程度：知道半透明物件不該留下這筆紀錄。之後直接用這個詞。

半透明物件如果也把自己的深度寫進去，排在它後面的物件會被整片判定為「被擋住」而不畫。畫面上就會出現方形的破洞。

（2）關掉之後為什麼還要自己指定順序。

關掉深度寫入之後，疊合結果就完全由繪製順序決定：後畫的蓋在先畫的上面。

Three.js 預設會依距離排序。但這個作品的物件每一幀都在換位置，預設排序在邊界情況會跳動。所以你要自己明確指定。

> **這一段就是「懂原理還是有用」的地方。** 你可以完全不懂深度緩衝也能用 Three.js。一直用到畫面出現破洞為止。那時候你會在網路上找到十個互相矛盾的答案，然後一個一個試。

💡 那段指令為什麼那樣寫

Prompt 裡「請把這兩個數字的關係寫在註解裡」是刻意的。

24.6 和 15.5 這種「兩個數字之間的關係」，最容易在後續修改中被拆掉。三個月後有人覺得淡出太早，把 15.5 調到 25。照片就開始憑空出現，而且不會有任何錯誤訊息。

寫在註解裡，下一個人才知道這兩個數字不能單獨動。

階段 9　讓介面顏色自己跟著背景亮度翻轉

📖 讀完約 8 分鐘　🛠️ 動手約 25 分鐘

這一步處理的是：□ 連續訊號　□ 離散事件　☑ 狀態　□ 畫面轉場

- **這一步一定要懂**：反色門檻從背景亮度算出來，不是十個場景各寫一次。文字層完全不需要知道 3D 的存在。
- **這一步可以先不懂**：線性色與螢幕色怎麼換算、CSS 顏色混合的語法。
- **做完後你要能用自己的話回答**：換一組照片，反色的時機為什麼會自己跟著變？

⚖️ 動手前先想

階段 4「顏色要用挑的還是算的」的延伸。切換門檻是你調到看起來剛好的數字，還是從背景亮度算出來的？

前者換十張照片就要重調，後者不用。

門檻本身也不用憑感覺訂。先問自己：最亮的暗場景和最暗的亮場景，亮度各是多少？門檻要落在兩者中間。這裡的亮度用**線性亮度**：把底色換算成實際光量後的亮度，0 是全黑，1 是全白。範例作品六個暗場景的底色，線性亮度都在 0.06 以下。四個亮場景都在 0.54 以上。門檻 0.42 落在中間。換成你的照片之後，貼這段量一次：

貼給 AI

```
請把 scenes.js 十個場景底色的線性亮度算出來，由暗排到亮列成表。
告訴我反色門檻落在哪兩個場景之間，離兩邊各差多少。
```

🛠️ 動手做

🧰 這一步的產出

```
src/ui/Hud.js    ← 畫面上的文字層
index.html       ← 新增：文字層的結構
styles.css       ← 新增：一組顏色變數，深色、淺色各一組值
```

掛在主迴圈的哪個位置：每一幀更新，只吃三個數字。

畫面上應該看到左邊時間、右邊場面名、下方時刻索引。捲到亮的場景時，整層文字會翻成深色。後面 🧭 有一張成品截圖可以對照。

📋 貼給 AI（9–1：文字層）

貼給 AI

你只需要修改：只有「畫面上要有」那五行的欄位名稱，換成你題材的說法。**線性色那一段照貼**，它是這個階段最容易踩的坑。

```
建立 src/ui/Hud.js，並在 index.html 加上它需要的結構。

這一層完全不需要知道 WebGL 的存在，只吃三個數字：
目前主導的場景編號、一圈走完的比例、背景亮度。

畫面上要有：
- 左邊：時刻與羅馬拼音
- 右邊：場面名（直書）
- 下方：時刻索引（十格，可以點）
- 左上：作品名
- 右上：目前第幾張 / 總數

用背景亮度決定文字要深色還是淺色，
在最外層切換一個 class，CSS 只準備一組變數，不要寫第二套樣式表。

注意一件事：THREE.Color 讀進來的是線性色，不是螢幕上的 sRGB。
請在註解裡說明這個差別，並讓亮度門檻在線性空間下是合理的數值。
```

點時刻索引要能跳到那個場景（用階段 7 做的那個方法）。

用中文註解說明每一段在做什麼。

📋 貼給 AI（9-2：色票的對比度）

貼給 AI

你只需要修改：沒有。整段照貼。

時刻索引的每一格用該場景的代表色。問題來了：
夜晚場景的代表色很深，在深色背景上等於看不見。

不要加邊框、不要加陰影。請用一個 `css` 的方法解決，
讓色票在深色和淺色背景下都看得見，而且仍然保有原來的色相。

做完之後解釋一下你用的方法，以及它為什麼比加邊框好。

✅ 怎麼知道做對了

- [ ] 從最暗的場景捲到最亮的場景，文字自動從淺色翻成深色
- [ ] **隨便改一組顏色資料，反色的時機跟著變**（這證明它是算出來的，不是寫死的）
- [ ] 十個色票在深色和淺色背景下都看得見
- [ ] 停在第 10 張時點時刻索引的第 1 格，畫面只往前走一張，**不是倒帶九張**
- [ ] 文字層的程式碼裡，沒有出現任何 Three.js 的東西

最後一項可以請 Claude 確認：

貼給 AI

檢查 `src/ui/Hud.js`，裡面有沒有任何地方引用到 `three` 或 `WebGL` 相關的東西？
有的話列出來。

正確答案是「沒有引用」。註解裡提到 WebGL 不算。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
反色時機怪怪的	亮度門檻用了螢幕顏色的直覺值	「反色的時機不對，請確認你算亮度時用的是線性色，門檻也要在線性空間」
文字被照片蓋住	圖層順序	「文字層應該永遠在照片上面」
點文字沒反應、點不到照片	文字層擋住了滑鼠	「文字層要設成不攔截滑鼠，只有可以點的按鈕才攔截」
點第 10 格倒退九張	階段 7–2 沒做或沒生效	回階段 7–2
色票在深色背景看不見	9–2 沒做	重跑 9–2

這一步失敗可以跳過嗎：色票對比（9–2）可以先跳過。文字層本身不行，階段 10 需要它。

📖 為什麼這樣做

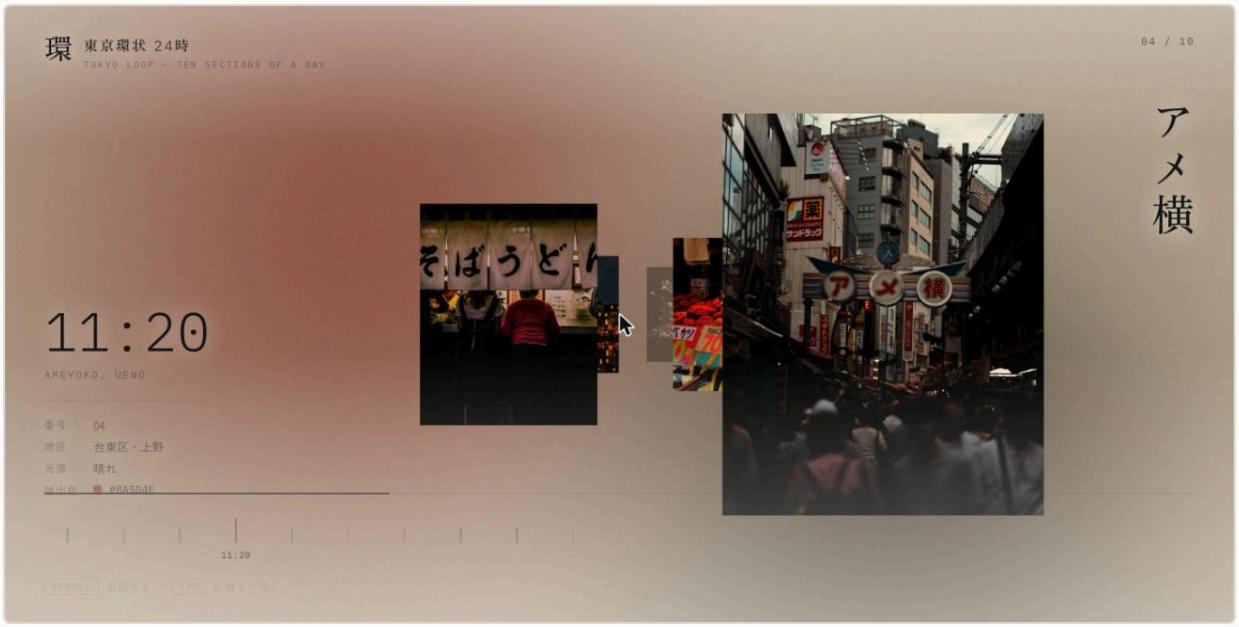
🎯 這一步的重點

文字什麼時候翻成深色，由背景亮度算出來，不是十個場景各寫一次。

🧭 這一步在做什麼

讓畫面上的文字顏色，自己跟著背景亮度切換。

重點是：**它是算出來的，不是寫死的。** 換一組照片，切換的時機會自己跟著變。



> **你要看的是什麼**：這張的背景是淺粉色（從那張阿美橫町的照片算出來的），而左邊的時間、下方的索引、右邊的直書標題，**全部自動變成深色**。沒有人在資料裡寫「這一張要用深色文字」，是背景亮度超過門檻，介面自己翻的。階段 4 之所以要真的去算顏色，就是為了這個。

🧩 它在建構流程的位置

```
藝廊 ──▶ 【介面文字層】
```

掛在主迴圈的哪個位置：每一幀更新，但它**完全不知道 3D 的存在**，只吃三個數字。

文字層只吃三個數字：現在是第幾個場景、走到幾成、背景多亮。**3D 畫面和文字層之間只傳三個數字**。所以文字層可以單獨測試、單獨改，3D 那邊出問題也波及不到它。

一個很多人踩的地方

你把一組色碼填進 `scenes.js`，程式讀出來的數字和你填進去的那個對不上。

Three.js 開啟色彩管理後，會自動把螢幕顏色轉成「線性色」。所以 `new THREE.Color('#d8cfc2').r` 讀出來**不是** `0.847`，是 `0.687`。

> **白話解釋**：螢幕顯示的顏色數值是經過「人眼感受」調整過的，而顯示卡做混色運算需要的是「物理上的光量」。兩者之間差一個次方曲線。 Three.js 幫你轉了，但你去讀那個值的時候拿到的是轉換後的數字。

如果亮度門檻是照「看起來的亮度」訂的，拿它切換文字顏色就會出錯。症狀是背景明明很亮，卻被判定成暗色。

不知道這件事，你會在幾個數字之間調半天，而且永遠不知道為什麼有幾個場景就是不對。

💡 那段指令為什麼那樣寫

第二個 Prompt 是一道小考題。

問題：時刻索引的每一格要用該場景的代表色。但夜晚場景的代表色很深，在深色背景上等於看不見。

限制：不要加邊框、不要加陰影，而且要保有原來的色相。

先回想一下 Claude 給你的做法，再往下看。

預期答案：用 CSS 的顏色混合，把場景代表色和目前的前景色混合。

為什麼這比加邊框好：邊框是在外面補一層東西，深色和淺色各要調一次。混色讓色票本身跟著主題變。深色模式往亮混，淺色模式往暗混。**一行 CSS 就涵蓋兩種背景，不用寫判斷分支**。

階段 10　點一張照片，讓它從原位飛到面板

📖 讀完約 10 分鐘　🛠️ 動手約 35 分鐘

這一步處理的是：□ 連續訊號　☑ 離散事件　□ 狀態　☑ 畫面轉場

- **這一步一定要懂**：判斷依據：起點和終點在不在同一套座標系統。這決定了要不要引入套件。
- **這一步可以先不懂**：FLIP 四個步驟的實作細節、樣式優先權的規則。順序照 Prompt 貼就好。
- **做完後你要能用自己的話回答**：為什麼這個轉場值得用套件，捲動的阻尼卻不值得？

⚖️ 動手前先想

這個轉場要自己寫還是用套件？

依據是「起點和終點在不在同一套座標系統」。

- 同一套（例如從左邊移到右邊）→ CSS 一行就好，不要用套件
- 兩套（這個案例）→ 用 Flip

這個判斷可以推廣：套件值得引入的時機，是它處理掉的細節，你自己處理要花好幾天。而且處理不完。

🛠️ 動手做

🧰 這一步的產出

```
src/ui/Detail.js    ← 點擊照片後展開的資訊面板
index.html          ← 新增：面板的結構
styles.css          ← 新增：面板的樣式
```

掛在主迴圈的哪個位置：不在迴圈裡。它由點擊觸發，用 GSAP 播完就結束。

📋 貼給 AI

貼給 AI

你只需要修改：沒有。整段照貼。**那三步的順序一個字都不要動**，換順序就是這個階段最常見的兩種失敗。

```
建立 src/ui/Detail.js，並在 index.html 加上面板的結構。
用 GSAP Flip 做展開轉場。

流程必須是這三步，順序不能換：
1. 把面板裡的圖片放到「起點矩形」（用直接寫在元素上的樣式），
   顯示出來，然後用 Flip 量測目前狀態
2. 清掉直接寫上去的樣式、加上展開用的 class，讓 CSS 決定終點版面
3. 用 Flip 倒推出動畫並播放

起點矩形請用階段 5 主迴圈提供的那個方法拿到。

同時要做的事：
- 把 3D 那一張的透明度設為 0，讓網頁元素接手（關閉時要還原）
- 凍結捲動
- 背景壓暗
- 面板文字一行一行依序進場
- Esc 和點背景都能關閉
- 滑鼠停在某張照片上時，只有那一張顏色變濃，游標變成一個標記（快速捲動時不判定）
- 停手 5 秒後浮出操作提示，動一下就淡回去（用階段 7 的「已經靜止幾秒」）

CSS 有一個必須遵守的規則：面板裡那張圖的位置和尺寸，
只能由「直接寫在元素上的樣式」（起點）或「展開用的 class」（終點）
其中一邊持有。不要在基礎樣式裡也寫位置和尺寸，
否則會和 Flip 的終點打架。

用中文註解說明每一段在做什麼，特別是那三步的順序為什麼不能換。
```

✅ 怎麼知道做對了

- [ ] 點任何一張照片，它是從**原本的位置**飛到面板，不是從畫面邊緣
- [ ] 關閉時飛回原位，而且 3D 那一張接得上（沒有閃一格）
- [ ] 展開的瞬間沒有看到兩張一樣的照片

- [ ] Esc 和點背景都能關閉
- [ ] 我說得出 FLIP 四個字母各代表什麼（見後面〈FLIP 是什麼〉那張表）

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
照片從畫面左上角飛過來	FLIP 三步的順序錯了	「照片是從左上角飛過來的，請檢查 Flip 的三步順序」
完全沒有動畫，直接跳	量測的時機不對	同上
動畫有跑，但終點位置不對	基礎樣式也寫了位置	「終點位置不對，請確認基礎 CSS 裡沒有寫位置和尺寸」
看到兩張一樣的照片	3D 那一張沒隱藏	「展開時會看到兩張，請把 3D 那一張的透明度設為 0」
中間閃一格	隱藏和動畫的順序反了	「中間會閃一下，請先畫一幀沒有那張照片的畫面，再開始動畫」
關閉後照片消失	沒有還原透明度	「關閉之後那張照片不見了，請確認有還原」

這一步失敗可以跳過嗎：可以。但「轉場要自己寫還是用套件」就只剩讀過、沒有做過。而這是這個案例唯一一個兩套座標系統的例子。建議至少讓它「能打開能關閉」，動畫不完美先往下。

📖 為什麼這樣做

🎯 這一步的重點

理解 FLIP 的原理，以及**為什麼這裡才是它該出場的地方**。

🧭 這一步在做什麼

點一張照片，它會從**原本的位置**飛到面板上。

這是整個作品裡唯一需要「兩種技術接在一起」的地方。



> **你要看的是什麼**：左邊那張照片，是從剛才畫面上它原本的位置飛過來的，不是從畫面邊緣滑進來。右邊的文字一行一行依序進場。背景整個壓暗，但沒有做成毛玻璃。**壓暗是為了讓面板上的照片和文字從背景裡分出來**。

🧩 它在建構流程的位置

> 藝廊 ＋ 介面 ⟶ 【展開面板】

掛在主迴圈的哪個位置：它不在主迴圈裡。它由點擊觸發，播完就結束，所以它是**離散事件（有明確的開始和結束）**，這正是它該用套件的理由。

FLIP 是什麼

四個步驟的縮寫：

步驟	做什麼
First	量測元素**現在**的位置
Last	直接套用**最終**樣式，讓瀏覽器把版面算完
Invert	算出「從最終位置回到原本位置」需要的變形，先套上去
Play	把這個變形動畫回 0

> **你要看懂的是什麼**：順序是「**先量起點 → 直接跳到終點 → 再倒著補動畫**」。聽起來很怪，但這是必要的，因為終點的位置是版面算出來的，動畫開始之前根本還不知道它會在哪。

為什麼不能直接對位置和大小做補間動畫？

因為終點是 CSS 版面決定的，例如「螢幕寬度的 34%，但最多 460px」。動畫開始前，你不知道它是多少像素。

為什麼這個作品特別適合示範 FLIP？

因為它的起點根本不在網頁的元素裡。起點是 3D 場景裡那張照片的四個角，經過相機投影之後得到的螢幕矩形。**那就是階段 5 主迴圈提供的第四件事**。

兩套完全不同的座標系統，中間沒有任何可以逐步過渡的共同數值。這正是 FLIP 存在的理由。

💡 那段指令為什麼那樣寫

Prompt 裡有三個要求，每一個都對應一種常見的失敗畫面。

(1)「順序不能換」。 這三步只要順序錯一步，就會得到「從左上角飛過來」或「完全沒有動畫」。這是 FLIP 最常見的兩種失敗。

(2)「位置和尺寸只能由一邊持有」。 直接寫在元素上的樣式，優先權高於樣式表裡的規則。所以如果基礎樣式也寫了位置，你必須清掉直接寫上去的那一份，終點的規則才會生效。

> 這種問題的症狀是「動畫有跑，但終點位置不對」，很難從動畫參數查出來，因為你會一直懷疑動畫參數，而問題其實出在樣式的優先權。

看到這個症狀，先不要調動畫參數，請 Claude 把兩邊攤開：

貼給 AI

```
請列出 styles.css 裡所有寫到面板圖片位置或尺寸的規則。
每一條標出它屬於基礎樣式，還是展開用的 class。先不要改。
```

兩邊都寫了位置或尺寸，問題就在那裡。

(3)「把 3D 那一張的透明度設為 0」。 不做的話，展開時會看到兩張一模一樣的照片疊在一起。而且順序要對：**先畫一幀「少了那一張」的畫面，再開始動畫**，否則中間會閃一格。

🧭 Checkpoint 3（做完階段 10）

作品已經完整可以互動了。先回答下面三題。

1. 指著 `Scroll.js` 說出：哪幾行是套件的工作，哪幾行是你的工作。
2. 為什麼回收距離一定要大於淡出完成的距離？
3. 詳細面板為什麼值得用套件，而捲動阻尼不值得？

> 第 1、3 題問的是同一件事：這個數值有明確的開始和結束，還是每一幀都在變？答案決定它交給套件，還是你自己算。第 2 題問的是階段 8 的兩個距離：照片要先完全看不見，才能被搬走。三題都答得出來就往下走，後面四個階段處理的是「能不能交付」。

階段 11　量一次效能，決定哪些其實不用改

📖 讀完約 12 分鐘　🛠️ 動手約 30 分鐘

這一步處理的是：☑ 連續訊號　□ 離散事件　□ 狀態　□ 畫面轉場

- **這一步一定要懂**：哪幾項優化該做，要先錄一次才知道。套件替你寫掉了程式，每一幀的計算量還是照算。
- **這一步可以先不懂**：開發者工具每一條線的意思、七個效能決定各自的技術原理。
- **做完後你要能用自己的話回答**：你有哪一項優化是量測之後決定「不做」的？依據是什麼？

⚖️ 動手前先想

這項優化值不值得做？

依據只有一個：**量過了嗎？**

先列清單、再量測、才決定要不要改。直接說「幫我優化效能」，AI 會把能做的全做一遍。其中包括讓程式碼變難讀、卻沒有效果的微優化。

這一步的驗收裡有一項是「至少有一項優化，是量測之後決定不做的」。列得出這一項，代表你真的量過。

🛠️ 動手做

🧰 這一步的產出

這一步不多出程式碼，只多**一份清單** `perf-notes.md`。它列出每一幀在做的事、各自的成本與優先順序，以及量測後決定不做的項目。

📋 貼給 AI（11–1：先列出來）

貼給 AI

你只需要修改：沒有。整段照貼，尤其「先不要改任何程式碼」。

請掃過整個專案，列出「每一幀」實際執行的工作。逐項列，先不要改任何程式碼。

每一項請標註：

- 一幀執行幾次
- 有沒有產生新的物件（會造成清理壓力）
- 有沒有讓瀏覽器被迫重算版面
- 在低階手機上是否可能成為瓶頸

最後排出優先順序：哪三項改了最有感？哪些其實不用管？

把結果存成 `perf-notes.md`。

📋 貼給 AI（11–2：真的量一次）

開發者工具要在 Chrome 或 Edge 裡打開，Browser 面板沒有。打開方法見階段 1 的〈🆘 如果沒有成功〉。

貼給 AI

你只需要修改：沒有。整段照貼。

請一步一步告訴我怎麼用 `Chrome` 的開發者工具量測，以及怎麼判讀。

我沒有做過，請從「怎麼打開」開始講。

我要量三件事：

1. 錄五秒的捲動，看有沒有掉幀
2. 看記憶體用量有沒有呈鋸齒狀（鋸齒代表每幀都在產生新物件）
3. 打開即時的 `FPS` 顯示，比較「靜止」和「快速捲動」兩種情況

量完之後，回頭對照我剛剛列的清單：

哪幾項是我猜的，哪幾項是真的？

把結論補進 `perf-notes.md`。

✅ 怎麼知道做對了

- [ ] 我手上有一份「每幀在做什麼」的清單
- [ ] 我實際打開過開發者工具，錄過一次
- [ ] **至少有一項優化是量測之後決定不做的**
- [ ] 我說得出解析度為什麼要限制在 2

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
開發者工具介面太複雜，不知道看哪	正常	「請告訴我畫面上哪一條是幀率，我要看的是哪個顏色」
錄出來的圖看不懂	正常	截圖貼給 Claude：「這是我錄到的，請幫我解讀」
完全沒掉幀，好像沒問題	你的電腦太好	開發者工具可以模擬低階裝置，請 Claude 教你怎麼開

這一步失敗可以跳過嗎：11–1 不行。階段 14 再量一次效能時，要拿這份清單對照。11–2 可以簡化，但**至少要打開一次開發者工具**。打開過一次，之後才知道要去哪裡看幀率。

📖 為什麼這樣做

🎯 這一步的重點

同一個效果在筆電上看起來一樣順。到了中階手機，每一幀多算的東西才會變成掉幀。

🧭 這一步在做什麼

把每一秒鐘電腦實際在做的事攤開來看。

🧩 它在建構流程的位置

功能全部做完之後，第一次回頭量效能。**太早做沒有意義**，你會優化到後來根本不存在的東西。

這個作品裡真實存在的七個效能決定

這七項是範例作品裡的決定，你自己的專案不一定每一項都有。**逐項讀。每一項都問自己：「不這樣做會怎樣？」**

#	決定	不這樣做會怎樣
1	解析度上限設成 2（階段 5）	高解析螢幕可能回報 3，要畫的像素量是 **2.25 倍**，等於讓顯示卡多做一倍多的工
2	顏色過渡用事先建好的暫存物件（階段 6）	每秒產生 180 個用完即丟的物件，瀏覽器清理時會週期性掉幀
3	速度低於門檻直接歸零（階段 7）	永遠停不下來，顯示卡每幀在算一個看不見的差異
4	半透明三件套（透明、關深度寫入、指定順序，階段 8）	照片交錯處出現方形破洞
5	淡出完成遠早於回收（階段 8）	看得到照片憑空出現
6	只對背景平面關掉「畫面外就不畫」的最佳化（階段 6）	全螢幕平面被誤判成「不在畫面內」而被跳過，背景整片消失
7	文字層設成不攔截滑鼠（階段 9）	文字區塊在照片上挖出一個點不到的洞

還有兩個功能，成本比看起來高：

這兩個都不是「寫錯了」，而是「**不知道那個功能的成本**」。

滑鼠偵測看起來只是一個函式呼叫。實際上它從滑鼠射一條線出去，和每個物件的每個面比一次。`gsap.to()` 看起來只是一行，實際上是在全域時間軸上註冊一個每一幀都會被走訪的物件。

滑鼠偵測的成本怎麼省？先問一題：快速捲動的時候，你需要知道滑鼠停在哪張照片上嗎？不需要。所以範例作品的 `main.js` 在換算成 0 到 1 的速度超過 0.25 時就不偵測。階段 10 Prompt 裡「快速捲動時不判定」那一句就是這樣來的。要確認你的版本：

貼給 AI

> 滑鼠偵測現在一幀跑幾次？快速捲動時有沒有跳過？請指出是哪幾行。

> 套件替你寫掉了滑鼠偵測和補間動畫的程式，但那些計算每一幀還是照算。

💡 那段指令為什麼那樣寫

第一個 Prompt 的重點是「**先列出來，不要改**」。列出來才知道哪幾項可以放掉。

第二個 Prompt 的重點是「回頭對照我剛剛列的清單：哪幾項是我猜的，哪幾項是真的？」

> **沒有錄過一次，清單上的每一項都還只是你猜的。**

階段 12　補回鍵盤操作、減少動態與觸控判斷

📖 讀完約 8 分鐘　🛠️ 動手約 30 分鐘

這一步處理的是：□ 連續訊號　☑ 離散事件　☑ 狀態　□ 畫面轉場

- **這一步一定要懂**：階段 7 關掉了原生捲動。鍵盤操作和「減少動態效果」要自己補回來。觸控裝置上滑動也會觸發「按下」，所以要在放開的時候才判斷。
- **這一步可以先不懂**：無障礙標記怎麼寫、10px 與 600 毫秒這兩個門檻是怎麼定出來的。
- **做完後你要能用自己的話回答**：手機上滑一下就彈出面板，為什麼滑鼠測一百次都測不出來？

🛠️ 動手做

🧰 這一步的產出

改動四個檔案：

- `src/main.js`：減少動態的判斷、輕點與滑動的區分、Enter 打開面板
- `src/input/Scroll.js`：方向鍵與 PageUp/PageDown
- `index.html`：無障礙標記
- `styles.css`：減少動態的樣式

📋 貼給 AI

貼給 AI

你只需要修改：沒有。整段照貼，五件事一件都不要刪。

```
階段 7 我阻止了瀏覽器的原生捲動，所以原生的鍵盤操作和
「減少動態效果」支援都沒了。請補上五件事：

1. 系統開啟「減少動態效果」時：速度訊號歸零、滑鼠效果直接到位、
   CSS 轉場縮到 0.15 秒。開啟之後不應該還有任何持續性的動畫
2. 鍵盤：方向鍵和 PageUp/PageDown 可以前後移動，Enter 打開目前這一張的面板，Esc 關閉
面板
3. 觸控：滑動不可以誤觸展開。
   請用「手指移動了多遠」和「按了多久」把輕點和滑動分開
```

```
4. 面板要標成對話框，打開時焦點要移到關閉鈕
5. 時刻索引的每個按鈕要有無障礙標籤

第 3 點請特別小心，說明你的判斷條件，以及為什麼選這兩個數值。
```

✅ 怎麼知道做對了

- [ ] 系統開啟「減少動態效果」後，畫面完全靜止，但十個場景仍然可以瀏覽
- [ ] 只用鍵盤可以走完全部十個場景，並且開關面板
- [ ] **拿真的手機滑十次，一次都不會誤開面板**
- [ ] 用鍵盤 Tab 走一遍，看得出目前焦點在哪裡

手機要打開你電腦上的網站，先貼這段：

貼給 AI

```
請讓本機伺服器可以從同一個 Wi-Fi 的手機打開，告訴我手機要輸入的網址。測完幫我關掉。
```

公司網路擋住的話，先跳過，階段 14 部署後再用手機開。

> **怎麼開「減少動態效果」**：Mac 是「系統設定 → 輔助使用 → 顯示器 → 減少動態效果」；Windows 是「設定 → 協助工具 → 視覺效果 → 動畫效果」。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
開了減少動態，畫面還在動	只處理了 CSS，沒處理 3D 那一層	「開啟減少動態之後，3D 那一層還在動，請一起處理」
手機上滑一下就彈出面板	用了「按下」而不是「放開」	「手機上滑動會誤開面板，請改成放開時判斷，加上移動距離和時間的門檻」
鍵盤按了沒反應	焦點不在頁面上	「請確認頁面載入後鍵盤事件有被接到」
沒有真的手機可以測	手邊沒有觸控裝置	開發者工具的手機模擬**測不出這個問題**。可以先跳過，但要在筆記上記一筆

這一步失敗可以跳過嗎：第 1、2 點不行（那是最基本的）。第 3 點如果沒有真機，可以先跳過，但階段 14 的驗收會再要求一次。

📖 為什麼這樣做

🎯 這一步的重點

原生捲動關掉之後，鍵盤操作和「減少動態效果」要自己補回來。

🧭 這一步在做什麼

讓不想看動畫的人、只能用鍵盤的人、用手機的人都能用。

階段 7 你阻止了瀏覽器的原生捲動。原生捲動本來就支援鍵盤和「減少動態效果」。你把它關掉了，這兩樣要自己補回來。

開頭那兩個打勾的格子是這樣來的。按一次方向鍵、點一下螢幕，都是離散事件（有明確的開始和結束）。系統有沒有開啟「減少動態效果」，是狀態。連續訊號那一格沒勾。這一階段做的是相反的事：使用者要求時，把連續訊號關掉。

🧩 它在建構流程的位置

效能之後、上線之前。

一個只有真機才測得出來的問題

在觸控裝置上，滑動捲動也會觸發「按下」事件。

如果用「按下」來開面板，使用者每滑一次就會彈出一次。

正確作法是在「放開」時判斷。用**手指移動的距離**和**按住的時間**，區分輕點和滑動。移動超過 10px 或按超過 600 毫秒，就當作滑動，不開面板。

> **這個問題用滑鼠完全測不出來。** 它是「只有真機才會發現」的典型代表，也是為什麼這個階段的驗收一定要拿真的手機。

💡 那段指令為什麼那樣寫

Prompt 的第 3 點要求「說明你的判斷條件，以及為什麼選這兩個數值」。

這是在防止 AI 給你一個沒有理由的數字。 10px 和 600 毫秒都有理由。10px 大約是手指自然抖動的範圍，600 毫秒大約是長按的門檻。它說得出理由，你才知道之後可不可以調。

要不要調，用你自己的手指量。手機上看不到 console，所以請它把數字顯示在畫面上：

貼給 AI

請暫時在每次手指放開時，把「移動了幾 px、按了幾毫秒」顯示在畫面左上角。

我要輕點十次、滑十次，看兩組數字各落在哪裡。看完再拿掉。

輕點的十次要全部在 10px 與 600 毫秒以內。滑動的十次，每一次至少要超過其中一個。

階段 13　盤點外部依賴，判斷能不能上線

📖 讀完約 6 分鐘　🛠️ 動手約 20 分鐘

這一步處理的是：□ 連續訊號　□ 離散事件　☑ 狀態　□ 畫面轉場

- **這一步一定要懂**：外部服務能不能留到正式環境，看它掛掉時畫面剩下什麼。做完這一步，你手上這個版本是 Learning Prototype。
- **這一步可以先不懂**：三個依賴各自要怎麼換掉。那是階段 14 的事。
- **做完後你要能用自己的話回答**：三個外部來源裡，哪一個失效最嚴重？為什麼？

⚖️ 動手前先想

這個外部服務可不可以留到正式環境？

階段 3 選用 importmap 時問過一次，這裡要回答它。

依據是：**它掛掉的時候，你的畫面剩下什麼。**

先猜：參考成品的照片、字體、程式套件各來自一個外部來源。三個各自掛掉時，畫面會剩下什麼？哪一個最危險？寫下來，做完斷網實驗再到〈📖 為什麼這樣做〉對照。

🛠️ 動手做

🧰 這一步的產出

一份依賴風險表，以及一個明確的決定：**這個版本要不要對外？如果要，哪三件事必須先換掉？**

📋 貼給 AI

貼給 AI

你只需要修改：沒有。整段照貼。尤其「不要只說建議自行託管」，那句話是在逼它給你具體代價。

```
上線前的檢查。請列出這個專案目前所有的外部依賴風險，
每一項給我「風險是什麼」和「正式環境該怎麼改」：

1. 圖片來源
```

2. 字體來源
3. 程式套件來源
4. 沒有打包步驟這件事本身

不要只說「建議自行託管」。請說明各自的具體代價：
多少 KB、多少次額外連線、失效時畫面會變成什麼樣。

把結果存成 deps-notes.md。

先看 `deps-notes.md` 列出的依賴，再做斷網實驗。開發者工具的打開方法見階段 1 的〈🆘 如果沒有成功〉。

貼給 AI

你只需要修改：沒有。整段照貼。

我要親眼看看依賴掛掉會怎樣。請一步一步告訴我：
怎麼在開發者工具裡把某一個網域擋掉，然後重新整理。

我想分別試三種情況：擋圖片、擋字體、擋程式套件。

✅ 怎麼知道做對了

- [ ] 我手上有一份依賴風險表
- [ ] **我親眼看過三種依賴各自失效時，畫面會變成什麼樣**
- [ ] 我說得出哪一個失效最嚴重、為什麼
- [ ] 我知道現在這個版本是 Learning Prototype，還不能對外

🆘 如果沒有成功

你可能會看到：不知道怎麼在開發者工具裡擋網域。

替代做法：直接把網路關掉，重新整理一次。三個會一起壞，但你看得到最糟的樣子。

這一步失敗可以跳過嗎：不行。階段 14 要換掉的外部依賴，都來自這份清單。

📖 為什麼這樣做

🎯 這一步的重點

看得出這個作品依賴誰，以及他們掛掉時你的畫面剩下什麼。

🧭 這一步在做什麼

上線前，列出這個作品從別人網站拿的每一樣東西。

這一步做完，你的作品是 Learning Prototype。還不能對外，但你會清楚知道為什麼不能，以及要補什麼。

開頭那個打勾的格子是「狀態」，指的是完成等級。這一階段不動畫面上的任何東西。它改變的是你對這個版本的判斷。你說得出它停在哪一級，也說得出三個來源各自掛掉時畫面剩下什麼。

🧩 它在建構流程的位置

正式版整理之前的盤點。階段 14 要做的每一件事，都來自這一步的清單。

斷網實驗的結果

參考成品有三個外部來源：

> 這是課程附件那份參考成品的情況。你照階段 4 做的版本，照片一開始就在本機。字體也可能沒有從字體服務載入，所以外部來源會比較少。以 `deps-notes.md` 實際列出的為準。你的版本沒有的那一項，斷網實驗改用課程附件那份做。

依賴	失效時畫面變成什麼樣	嚴重度
圖庫網站	十個平面全空，只剩背景色	高
字體服務	掉回系統字體，直書的字距會跑掉	中
程式套件來源	**整個作品不會啟動，停在載入畫面的 000**	最高

第三個是最危險的。它一失效，整個作品停在 000 不會啟動，而且你控制不了對方什麼時候出問題。

「沒有打包步驟」本身的代價

- 每個使用者都要多做幾次額外的連線
- 無法移除沒用到的程式碼（Three.js 完整版約 750KB，這個作品只用到階段 2 列的那十個東西）

教學用它沒問題：零安裝、打開就跑。撞到上面這兩項才要換掉，當初選它沒有錯。

回頭看用過的每一個套件

盤點完外部來源，順便把這個作品用到的套件也過一次：每一件事當初該交給誰做。

你想做的事	該用什麼	不該用什麼	為什麼
元素移動、縮放、淡入淡出	CSS	Three.js、GSAP	瀏覽器原生就在最快的路徑上跑，成本幾乎是零
幾個元素有先後順序的編排	GSAP 時間軸	自己寫計時器	暫停、倒轉、跳到某一秒，自己寫等於重做一遍
版面 A 變成版面 B 的轉場	GSAP Flip	對位置做補間	終點由 CSS 決定，動畫開始前你不知道它是多少
把不同裝置的滑動量換算成同一個單位	GSAP Observer	自己接事件	跨裝置差異太大，自己寫測不完
連續訊號（每一幀都在變：阻尼、視差、速度）	**自己寫**	補間動畫	補間會疊加、互相覆寫，而且每幀都被走訪
多個物件要共用同一台相機	Three.js	CSS 3D	CSS 的透視是各算各的
逐像素效果（混色、扭曲、顆粒）	shader	CSS 濾鏡	這是顯示卡的工作
幾十個以上的動態物件	Three.js	網頁元素	瀏覽器要另外存幾張圖，每一張都佔記憶體
捲動到某個位置觸發動畫	GSAP ScrollTrigger	自己算位置	但**先問一句**：真的需要嗎？

還有一列不在表上：

> 不用套件也做得到的效果，多引入一個套件有兩個代價。使用者要多載一次，你要多維護一個版本號。零依賴不代表品質比較低。一頁式的品牌網站，只用 HTML 和 CSS 就做得出來。

階段 14　把教學版整理成可以交付的正式版

📖 讀完約 10 分鐘　🛠️ 動手約 60 分鐘

這一步處理的是：□ 連續訊號　□ 離散事件　☑ 狀態　□ 畫面轉場

- **這一步一定要懂**：階段 3 選 importmap 是對的，這一步換成 Vite 打包。一次只做一項，每項做完先確認沒壞。
- **這一步可以先不懂**：Vite 的設定怎麼寫、打包怎麼移除沒用到的程式碼。
- **做完後你要能用自己的話回答**：你是依據哪幾件事，判斷 importmap 該換掉了？

🛠️ 動手做

🧰 這一步的產出

```
package.json          ← 依賴改成正常管理
vite.config.js        ← 打包設定
public/photos/        ← 壓縮過的照片
public/fonts/         ← 自己託管的字體
dist/                 ← 打包後的成品
```

以及一個實際可以打開的網址。

📋 貼給 AI（14-1：先要計畫）

貼給 AI

你只需要修改：沒有。整段照貼。**尤其最後兩句「先不要動任何檔案」和「一次做一項」。**

> 我要把這個教學版整理成可以正式交付的版本。
>
> 依據是我剛剛做的 `deps-notes.md`。
>
> 先不要動任何檔案。請給我一份計畫，包含這十件事的順序與作法：
>
> 1. 把圖片改成本地檔案（我會提供自己的照片）
> 2. 處理圖片尺寸與壓縮
> 3. 字體改成自己託管

4. 把 importmap 的套件改成正常的依賴管理
5. 導入 Vite 或其他合適的打包工具
6. 執行正式打包
7. 部署
8. 用真的手機測試
9. 再跑一次效能量測
10. 再做一次外部依賴檢查

每一項請告訴我：要做什麼、大概多久、做完怎麼確認沒壞。
如果某一項的順序不能換，說明為什麼。

我確認之後，我們一次做一項。

第 1 項「把圖片改成本地檔案」：照階段 4 做的話已經完成，確認後跳過。

📋 貼給 AI（14-2：一次一項）

計畫確認之後，一項一項來。每做完一項就問：

貼給 AI

你只需要修改：沒有。整段照貼，每做完一項就貼一次。

這一項做完了。請幫我確認網站還是正常的：
- 打包成功
- 打開之後十張照片都在
- 捲動、點擊、關閉都正常
- console 沒有錯誤

確認完之後我們再做下一項。

📋 貼給 AI（14-3：部署）

還沒接上 GitHub 的話，先照〈開始之前〉的〈接上 GitHub〉做一次。

> **名詞**：**Vercel** 是把 GitHub 上的專案變成公開網址的服務。這一步會拿到一個 `xxx.vercel.app` 網址。
> 你需要知道到什麼程度：知道接上之後，你每次推上 GitHub，它會自己重新部署。

> 貼給 AI
>
> **你只需要修改**：沒有。整段照貼，尤其最後那句「先把你準備執行的每一個指令列出來」。
>
> 準備部署。請確認打包產出是完整的，然後幫我做這三件事：
> - 把所有改動做一次 commit
> - 建一個 GitHub 專案，設成 private（Vercel 讀得到私人專案）
> - 把程式碼推上去
>
> 但先不要動手。先把你準備執行的每一個指令列出來給我看，
> 每一行說明它會做什麼、會不會產生我收不回來的東西，我確認之後你再做。

> Claude 先列指令、你確認、它才執行。推上去之後，你自己去 Vercel 把這個 GitHub 專案接上去，見下一節。

你自己做（14-4：把 Vercel 接上這個專案）

這一步只做一次。接上之後，**你每次推上 GitHub，Vercel 都會自己重新部署。不用再回來按任何東西。**

1. 到 vercel.com，用 GitHub 帳號登入
2. 選 **Add New Project**
3. 在列表裡找到你剛剛建的專案，把它匯入
4. 設定畫面不用改，Vercel 會自己認出專案用的打包工具
5. 等一到兩分鐘，會出現一個 `xxx.vercel.app` 的網址

> **畫面和上面寫的不一樣**：這類服務的介面常常改版。把你看到的畫面截圖貼給 Claude，說「我要把這個 GitHub 專案接到 Vercel 上，現在該點哪裡」，讓它帶你走。

接好之後的 `xxx.vercel.app` 網址，14-5 要用它。

📋 貼給 AI（14–5：最終驗收）

貼給 AI

你只需要修改：只有 `（貼上你的網址）`，換成你的網址。三個問題照貼。

網站已經上線在 **（貼上你的網址）**。請幫我做最後驗收：

1. 打包後的總檔案大小是多少？和之前用 `importmap` 的版本比，差多少？
2. 頁面載入時，一共發了幾個對外的請求？分別是什麼？
3. 還有沒有任何一個資源是從別人的網站來的？

第 3 點如果答案不是「沒有」，請列出來，並告訴我為什麼還留著。

✅ 怎麼知道做對了

- [ ] 打包成功，產出資料夾是完整的
- [ ] 打開開發者工具的 Network 分頁重新整理，所有請求都來自你自己的網址，沒有別的網域。打開方法見階段 1 的〈🆘 如果沒有成功〉
- [ ] 正式網址打得開，十張照片都在
- [ ] **我用真的手機開過一次**，滑動不會誤開面板
- [ ] 我再量過一次效能，和之前的紀錄有對照
- [ ] 檔案大小比教學版小（移除沒用到的程式碼的效果）
- [ ] 我說得出「什麼時候該把 importmap 換成打包工具」

全部打勾之後，這個作品是 Production-ready Baseline。

如果有任何一項沒做完

在你的筆記上明確寫下來：

```
目前仍是 Learning Prototype，尚未完成的項目：
- （列出來）
```

不要對外宣稱它完成了。 少做的那幾項，都是別人打開你的網站時會直接遇到的。

🆘 如果沒有成功

你可能會看到	通常是因為	怎麼辦
導入打包工具後畫面全黑	路徑改變了	「導入 Vite 之後畫面全黑，請檢查資源的路徑」
照片載不到	照片路徑寫成 `public/photos/…`，打包後要改成 `/photos/…`	「請告訴我圖片應該放在哪個資料夾才會被打包進去」
打包後檔案反而更大	沒有正確移除未使用的程式碼	「請確認打包有移除沒用到的程式碼，並告訴我怎麼確認」
字體變成系統字體	字體檔沒被打包	「字體沒有生效，請檢查字體檔的路徑和格式」
部署後空白	輸出目錄設定不對	把部署服務的錯誤訊息整段貼給 Claude

這一步失敗可以跳過嗎：可以，但那樣你的作品就停在 Learning Prototype。至少要做完第 1、2、4、5 項（圖片本地化 + 打包），那是差別最大的四項。

📖 為什麼這樣做

🎯 這一步的重點

把階段 13 風險表上的三項換掉，做出一個打包過、部署在公開網址的版本。

🧭 這一步在做什麼

把階段 13 列出來的問題，一項一項換掉。

這一階段處理的是從「會動」到「能交出去」之間的那幾件事。

🧩 它在建構流程的位置

流程的終點。做完這一步，作品從 Learning Prototype 變成 Production-ready Baseline。

開頭那個打勾的格子是「狀態」，指的就是這件事。這一階段改變的是作品的完成等級。畫面上不會多出任何東西，使用者看到的和階段 13 一模一樣。

為什麼不一開始就這樣做

因為打包工具會讓每一次修改多一個步驟，而學習階段你需要的是「改一下、馬上看到」。

這就是為什麼階段 3 選 importmap 不算選錯。要「改一下、馬上看到」時，importmap 是對的。要壓檔案大小、要自己託管時，它就不夠用了。

> 💡 **那段指令為什麼那樣寫**
>
> 這個階段有十件事要做，所以第一個 Prompt 的重點是「**先給我計畫，不要動手**」。
>
> 一次做十件事，出錯了你會找不到是哪一件造成的。正確的做法是：先列計畫 → 一次做一項 → 每一項做完確認網站還會動 → 再下一項。

完成清單

- [ ] 我完成了課前作業（十張照片），也在階段 1 填完狀態表
- [ ] 十四個階段的 ✅ 驗收都通過了
- [ ] 打包成功，`console` 沒有錯誤
- [ ] 我用真的手機開過（不是開發者工具的模擬）
- [ ] 我知道目前的成果是哪一級，以及還差什麼
- [ ] 我至少有一項優化是量測之後決定不做的
- [ ] 我親眼看過依賴失效時畫面會變成什麼樣
- [ ] 我可以用自己的話說出：**這個作品裡的每一個套件，各自在解決什麼問題**

離開這個範例，我會不會自己做？

回顧

#	問題	對應階段
1	這個專案裡，哪一個效果非用 WebGL 不可？哪一個其實 CSS 就夠？	2
2	為什麼捲動的阻尼不能用補間動畫？	2、7
3	你的背景顏色是從哪裡來的？換十張照片要改幾個檔案？	4、6
4	為什麼回收距離一定要大於淡出距離？	8
5	介面的深淺色是怎麼決定的？寫死的還是算出來的？	9
6	FLIP 四個字母各代表什麼？為什麼不能直接對位置做補間？	10
7	你的專案每一幀產生幾個新物件？你怎麼知道的？	11
8	阻止原生捲動之後，你補了哪些無障礙的東西？	12
9	什麼時候你會把 importmap 換成打包工具？	3、13、14

遷移

#	問題
10	你想做一個跟著滑鼠移動的背景。你會怎麼判斷 CSS、GSAP、Three.js 哪一個值得用？
11	客戶說「首頁要有一個很有質感的動畫」。你會先問他哪三個問題？
12	你接手一個別人寫的專案，發現它每一幀都在建立新物件。你會怎麼向不懂技術的人說明這件事的影響？
13	你的下一個專案要用一個沒用過的套件。在寫第一行程式碼之前，你會先問 AI 哪三件事？
14	這個作品把動畫全部拿掉之後，還剩下什麼？如果答案是「什麼都沒有」，你會怎麼改？

第 10 題答不出來的話，回去看階段 2。

常見卡關

症狀	原因	解法
打開一片空白，console 出現 CORS	用檔案方式打開了	請 Claude 起一個本機伺服器
外掛註冊了但沒效果，也沒有錯誤訊息	importmap 載到兩份 gsap	全部指向同一個來源的同一個版本
讀不出照片的像素	跨網域的圖片不能讀像素	用本機檔案，或加上 `crossOrigin='anonymous'`
照片交錯處有方形破洞	深度寫入沒關	半透明三件套一起設（階段 8–2）
背景整片消失	全螢幕平面被「畫面外就不畫」誤判	對背景平面關掉那個最佳化
展開動畫終點位置不對	基礎樣式也寫了位置	位置只能由一邊持有（階段 10）
畫面上只看得到一張照片	橫向位置太集中	拉開各張照片的橫向間距
直書標題被照片蓋住	橫向位置的極值超出安全範圍	請 Claude 反推安全範圍
手機滑動會誤開面板	用了「按下」	改用「放開」，加上距離和時間門檻（階段 12）
亮度判斷在某幾個場景就是不對	讀到的是線性色	門檻要在線性空間下訂（階段 9）
停手後畫面還在微微發亮	速度沒有歸零門檻	加上門檻（階段 7）
分頁切到背景後動畫停住	瀏覽器的正常行為	不用修
手機發燙、掉幀	解析度沒限制、貼圖沒壓縮	階段 11 的第 1 項、階段 14 的圖片壓縮

症狀	原因	解法
卡超過 20 分鐘	原因還沒找到	整段錯誤訊息貼給 Claude，再加一句：「請先不要修改任何東西。先告訴我最可能的三個原因，以及你準備先檢查哪一個。」還是不行就先跳過，每個階段的 🆘 都寫了能不能跳過

為什麼「懂原理」還是重要

這個作品裡有八個地方，不懂就會卡住，而且卡住的時候，你連問題該怎麼描述都不知道。

現象	真正的原因	不懂的話會怎麼處理
照片交錯處出現方形破洞	半透明物件寫了深度	以為是圖檔壞了，換一張再試
亮度判斷在某幾個場景就是不對	讀出來的是線性色	調數字調到放棄
觸控板很順，滑鼠一格跳三張	滾輪事件的單位跨裝置不一致	「在我電腦上是好的」
手機上滑一下就彈出面板	滑動也會觸發「按下」	桌機測一百次都測不出來
展開動畫終點位置不對	樣式優先權	一直改動畫參數，但問題在 CSS
點第 10 格卻倒帶九張	會循環的位置刻度沒有走短邊	以為是動畫太慢，把時間調短
捲動停下來後畫面還在微微發亮	平滑演算法永遠到不了目標	看不出來，直到有人問「為什麼有點閃」
手機發燙、掉幀	解析度沒限制、貼圖沒壓縮	「WebGL 就是很吃效能」

這八項的共通點：它們都不是「AI 寫錯了」。

這八項都是在某個你沒有指定的條件下才出問題，例如手機觸控或線性色。你要在 Prompt 裡寫出那個條件，Claude 才會處理。

> 套件沒有替你決定上面這八件事的**成本**和**前提**。這八個現象出現時，你要認得出是哪一項，才知道該把哪一句貼給 Claude。

名詞白話對照表

動畫與瀏覽器

名詞	需要知道到什麼程度
主迴圈（正式名稱 requestAnimationFrame，簡稱 rAF）	瀏覽器每畫一張新畫面之前呼叫你一次。一般螢幕一秒 60 次
一幀	螢幕上的一張畫面。60fps 代表每 16.7 毫秒要畫完一張
掉幀	某一張畫面來不及在 16.7 毫秒內畫完，使用者會看到頓一下
lerp（線性插值）	「每一幀往目標靠近一點點」。這個案例所有的平滑都是它做的
補間動畫（tween）	給定起點、終點、時間長度，由套件自動算出中間每一幀
時間軸（timeline）	把多個補間排出先後順序的容器，可以整體暫停、倒轉
加速曲線（easing）	動畫快慢的變化方式，例如「先慢、中間快、最後慢」
重算版面（reflow）	瀏覽器被迫立刻重新計算整個版面。會卡住主執行緒
圖層	瀏覽器把某個元素單獨存成一張圖，之後移動它就不用重畫。加速但吃記憶體
記憶體清理（GC）	瀏覽器自動清掉沒人再用的物件。清的時候會停一下，所以要少產生用完即丟的東西

3D 與顯示卡

名詞	需要知道到什麼程度
Three.js	一套在瀏覽器裡畫 3D 的程式庫。這個案例用它管相機和照片平面
GSAP	一套做網頁動畫的程式庫。這個案例只用它的 Observer、Flip 和時間軸
開發者工具	瀏覽器內建的檢查面板，按 F12（Mac 是 Option + Command + I）打開。這個案例用它的 Network 和 Performance 分頁
WebGL	瀏覽器裡直接操作顯示卡的介面。Three.js 是它的包裝
場景	一個容器，裝著所有要畫的東西
相機	決定從哪裡看、看多廣

名詞	需要知道到什麼程度
shader（著色器）	一小段跑在顯示卡上的程式。這個案例用它處理每個像素都要一起變化的效果，你不需要讀懂
uniform	從 JavaScript 傳給 shader 的數字。每一幀可以直接覆寫
貼圖	貼在物件表面的圖片
深度緩衝	記錄每個像素目前最近的物件有多遠，用來讓後面的東西不要蓋住前面的
繪製順序	半透明物件關掉深度寫入之後，就完全靠這個決定誰蓋誰
滑鼠偵測（raycast）	從滑鼠射一條線出去，逐一比對它打到哪個面。每個物件的每個面都要比一次，成本比看起來高
畫面外就不畫（frustum culling）	不在相機視野裡的東西就不畫。全螢幕平面容易被誤判
解析度倍率（dpr）	一個點實際上要畫幾個實體像素。高解析螢幕是 2 或 3
線性色 vs 螢幕色	螢幕色是給人眼看的數值，線性色是物理光量。混色運算要用後者
色帶	純色漸層在暗處出現的階梯。加一點雜訊可以打散

工具與流程

名詞	需要知道到什麼程度
console	瀏覽器裡顯示錯誤訊息的那一頁。出錯時整段紅字複製起來貼給 AI，不用看懂
CSS	網頁本來就有的樣式規則，管顏色、位置、大小、轉場。瀏覽器內建，使用者不用多載
importmap	寫在 HTML 裡的對照表，告訴瀏覽器某個套件要去哪個網址拿。有它就不用打包工具
打包工具（Vite）	把所有程式碼和資源合併、壓縮、轉譯成瀏覽器看得懂的檔案
移除未使用程式碼	打包時把沒被用到的部分丟掉，減少檔案大小。importmap 做不到
CDN	分散在各地的檔案伺服器，離使用者近所以快。缺點是你控制不了它掛掉
CORS	瀏覽器的跨網域安全限制。用檔案方式打開會被它擋下來
ES Module	JavaScript 把程式碼拆成多個檔案、互相引用的標準寫法。程式碼裡的 `import` 就是它

名詞	需要知道到什麼程度
`crossOrigin`	載入別的網站的圖片時要加的一個設定。不加的話，瀏覽器不准你讀出那張圖的像素
會循環的位置刻度（繞圈座標）	走到底會接回開頭的一把位置刻度。所有計算都要取餘數，而且要注意負數

各階段預估時間

階段	主題	讀完	動手
課前	開始之前（含課前作業）	20 分	60 分
1	先看成品，觀察三件事、填狀態表	8 分	35 分
2	建立判斷框架：離散與連續	12 分	25 分
3	建立不用安裝就能跑的專案	8 分	20 分
4	準備照片，從照片算出背景顏色	10 分	40 分
5	建立主迴圈	8 分	25 分
6	做背景：顏色跟著捲動變	8 分	25 分
7	做捲動訊號，把分界線寫成檢查	15 分	50 分
8	讓照片無限循環，看不出接縫	10 分	35 分
9	讓介面顏色自己翻轉	8 分	25 分
10	點一張照片，讓它從原位飛出	10 分	35 分
11	量一次效能	12 分	30 分
12	補回鍵盤、減少動態與觸控	8 分	30 分
13	盤點外部依賴	6 分	20 分
14	整理成可以交付的正式版	10 分	60 分
	合計	**約 2.6 小時**	**約 8.6 小時**

時間有限的話，可以先跳過階段 14，之後再回來補（但要記得標明作品還是 Learning Prototype）。 **不要跳過階段 2 和階段 7**。跳掉它們，這個案例就只剩照著貼指令，沒有做過任何一次選型判斷。

合計超過十小時，建議切成三次：

分段	階段	產出
第一次	開始之前 → 階段 6	資料層、主迴圈、會變色的背景
第二次	階段 7 → 階段 10	完整可以互動的作品
第三次	階段 11 → 階段 14	效能、無障礙、正式版